BAB 5

IMPLEMENTASI

Pada bab ini akan dibahas tahapan implementasi sistem, mulai dari proses perencanaan hingga tahap penggunaan, sesuai dengan rancangan desain sistem yang telah dijelaskan pada bab sebelumnya. Implementasi sistem ini mencakup pengembangan basis data (database), kode program, serta antarmuka pengguna (User Interface/UI) yang digunakan dalam sistem tugas akhir. Tahapan implementasi ini bertujuan untuk memastikan bahwa sistem dapat berjalan sesuai dengan kebutuhan pengguna dan mendukung seluruh proses pengelolaan properti, lokasi acara, tenant acara, serta transaksi yang ada secara terpadu.

5.1 Implementasi Basis Data

Bagian ini menjelaskan mengenai basis data yang digunakan untuk menyimpan seluruh data yang diperlukan dalam sistem tugas akhir. Perancangan basis data dilakukan menggunakan pendekatan Entity Relationship Diagram (ERD) sebagai langkah awal untuk memvisualisasikan hubungan antar entitas yang terlibat, sehingga struktur data dapat tersusun secara sistematis dan efisien. Implementasi basis data dilakukan menggunakan MySQL, sedangkan proses integrasi basis data dengan aplikasi web memanfaatkan framework Laravel versi 11.46.1. Setelah perancangan ERD selesai, implementasi dilakukan dengan membuat tabel-tabel sesuai kebutuhan sistem melalui migrasi Laravel, dengan total sebanyak 27 tabel yang saling terintegrasi untuk mendukung seluruh fungsi sistem, dan hasilnya dapat dilihat menggunakan phpMyAdmin, yaitu alat berbasis web untuk pengelolaan MySQL.

Struktur basis data yang telah diimplementasikan dalam sistem ini disajikan secara visual pada Gambar 5.1, untuk memberikan gambaran yang jelas mengenai pengaturan tabel serta hubungan antar data di dalam sistem.

Table	Action	Rows	Type	Collation	Size	Overhead
foto_acara	Browse Structure Search Insert Empty Drop	7	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
foto_lokasi	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
foto_properti	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
histori_lokasi	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
histori_properti	Browse Structure Search Insert Empty Drop	15	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
karyawan	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
kategori	Browse Structure Search Insert Empty Drop	10	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
kategori_perubahan	Browse Structure Search Insert Empty Drop	26	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
kategori_perubahan_lokasi	Browse Structure Search Insert Empty Drop	17	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
kategori_tenant	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8_general_ci	16.0 KiB	-
kategori_tenant_lokasi	Browse Structure Search Insert Empty Drop	9	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
kunjungan	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
lokasi	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
lokasi_kategori	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
pembayaran_acara	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
pembayaran_properti	Browse Structure Search Insert Empty Drop	15	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
pembayaran_tenant	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
pengajuan_aktivasi	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
pengguna	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
properti	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
properti_aktif	Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
tawaran	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
tenant_daftar	Browse Structure Search Insert Empty Drop	1	InnoDB	utf8_general_ci	48.0 KiB	-
tenant_lokasi	Browse Structure Search Insert Empty Drop	5	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
transaksi_acara	Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_unicode_ci	80.0 KiB	-
transaksi_properti	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	64.0 KiB	-
video_properti	Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
27 table(s)	Sum	160	InnoDB	utf8_general_ci	1.1 MiB	0 B

Gambar 5.1 Struktur *Database* Sistem

Setiap tabel yang dibuat memiliki struktur yang telah dirancang sesuai dengan desain data yang dijelaskan pada bab sebelumnya. Berikut ini merupakan penjelasan mengenai struktur beberapa tabel yang telah diimplementasikan dalam sistem:

5.1.1 Implementasi Basis Data Pengelolaan Data Pengguna

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung pengelolaan data pengguna dalam sistem informasi jual beli dan sewa

properti di kawasan Kota Lama Surabaya. Implementasi basis data ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Pengelolaan Data Pengguna yang telah dijelaskan pada Bab 4.1.1. Pada subbab ini dijelaskan struktur tabel yang digunakan beserta fungsi setiap atribut dalam mendukung proses pengelolaan akun pengguna pada sistem.

a. Tabel karyawan

Tabel karyawan digunakan untuk menyimpan informasi mengenai pegawai atau petugas yang berperan dalam pengelolaan sistem informasi jual beli dan sewa properti. Atribut yang terdapat pada tabel ini terdiri dari karyawan_id sebagai *primary key* yang berfungsi untuk membedakan setiap data karyawan, nama yang menyimpan nama lengkap karyawan, email yang digunakan untuk proses autentikasi, *password* yang menyimpan kata sandi akun karyawan, serta no_hp yang digunakan untuk menyimpan nomor *handphone* karyawan. Selain itu, terdapat atribut *created_at* dan *updated_at* yang berfungsi untuk mencatat waktu pembuatan serta pembaruan data karyawan secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**karyawan_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**nama**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
3	**email**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**password**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
5	**no_hp**	varchar(15)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
6	**created_at**	timestamp			Yes	*NULL*			Change Drop More
7	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.2 Implementasi Tabel karyawan

b. Tabel pengguna

Tabel pengguna digunakan untuk menyimpan data seluruh akun yang dapat mengakses sistem, baik sebagai investor, pemilik properti, pengelola lokasi, maupun pihak lainnya yang terlibat dalam pengelolaan sistem. Atribut yang terdapat pada tabel ini terdiri dari pengguna_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap pengguna, karyawan_id sebagai *foreign key* yang menghubungkan pengguna dengan data karyawan terkait, nama untuk menyimpan nama lengkap pengguna, email sebagai alamat surat elektronik yang juga digunakan dalam proses autentikasi, serta password untuk menyimpan kata sandi akun pengguna.

Selain itu, terdapat atribut tanggal_persetujuan yang digunakan untuk mencatat waktu verifikasi akun oleh admin dinas, no_hp untuk menyimpan nomor handphone pengguna, dan role yang berisi nilai enum seperti “Investor”, “Pemilik Properti”, dan “Pengelola Lokasi” untuk menentukan hak akses pengguna dalam sistem.Atribut status digunakan untuk menunjukkan kondisi akun, dengan pilihan nilai enum yaitu “Aktif”, “Menunggu”, dan “Ditolak”. Tabel ini juga memiliki atribut *created_at* dan *updated_at* yang mencatat waktu pembuatan dan pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**karyawan_id**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
3	**nama**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**email**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
5	**password**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
6	**tanggal_persetujuan**	datetime			Yes	*NULL*			Change Drop More
7	**no_hp**	varchar(15)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
8	**role**	enum('Investor', 'Pemilik Properti', 'Pengelola Lo...	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
9	**status**	enum('Aktif', 'Menunggu', 'Ditolak')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
10	**created_at**	timestamp			Yes	*NULL*			Change Drop More
11	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.3 Implementasi Tabel pengguna

5.1.2 Implementasi Basis Data Pengelolaan Data Properti dan Aktivasi

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung pengelolaan data properti dan proses aktivasi properti dalam sistem informasi jual beli dan sewa properti di kawasan Kota Lama Surabaya. Implementasi basis data ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Pengelolaan Data Properti dan Aktivasi yang telah dijelaskan pada Bab 4.1.2. Pada subbab ini dijelaskan struktur tabel-tabel yang berperan dalam penyimpanan data properti, media pendukung, status aktivasi, serta riwayat perubahan data properti yang dilakukan oleh pengguna maupun karyawan.

a. Tabel properti

   Tabel properti digunakan untuk menyimpan seluruh data properti yang terdaftar di dalam sistem. Tabel ini menjadi salah satu tabel utama dalam sistem karena berfungsi untuk mencatat informasi detail mengenai setiap properti yang dimiliki atau dikelola oleh pengguna. Atribut yang terdapat pada tabel ini terdiri dari properti_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap data properti, serta pengguna_id dan karyawan_id sebagai *foreign key* yang menghubungkan data properti dengan pengguna dan karyawan terkait. Atribut nama_properti digunakan untuk menyimpan nama properti, sedangkan deskripsi berisi penjelasan umum mengenai kondisi dan karakteristik properti. Terdapat pula atribut gaya_arsitektur yang menjelaskan konsep arsitektur bangunan, serta jenis_komersial dengan nilai enum seperti “Jual”, “Sewa” untuk menandai jenis transaksi properti. Atribut nomor_shm,

nama_pemilik, dan alamat_lengkap menyimpan informasi legalitas dan lokasi properti. Koordinat lokasi disimpan dalam atribut latitude dan longitude untuk kebutuhan pemetaan digital.

Selain itu, atribut harga, luas_bangunan, luas_tanah, tahun_pendirian, renovasi_terakhir, jumlah_ruangan, dan kondisi_bangunan digunakan untuk menggambarkan detail fisik dan nilai komersial dari properti. Beberapa atribut tambahan seperti listrik, parkir, dan air_pam menunjukkan fasilitas yang tersedia, sedangkan fasilitas_tambahan berisi deskripsi fitur khusus lainnya. Atribut status menunjukkan kondisi ketersediaan data dengan nilai enum "Aktif", "Nonaktif", atau "Dihapus", sedangkan status_acc menyimpan status persetujuan properti dengan nilai "Diterima", "Ditolak", atau "Diajukan". Atribut tanggal_persetujuan digunakan untuk mencatat waktu validasi properti oleh admin dinas, dan alasan berfungsi untuk menyimpan keterangan tambahan saat terjadi penolakan atau pembatalan. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**properti_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**karyawan_id**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
4	**nama_properti**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
5	**deskripsi**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
6	**gaya_arsitekstur**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
7	**jenis_komersial**	enum('Jual', 'Sewa')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
8	**nomor_shm**	varchar(20)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
9	**nama_pemilik**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
10	**alamat_lengkap**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
11	**latitude**	decimal(10,7)			Yes	*NULL*			Change Drop More
12	**longitude**	decimal(10,7)			Yes	*NULL*			Change Drop More
13	**harga**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
14	**luas_bangunan**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
15	**luas_tanah**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
16	**tahun_pendirian**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
17	**renovasi_terakhir**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
18	**jumlah_ruangan**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
19	**kondisi_bangunan**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
20	**listrik**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
21	**parkir**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
22	**air_pam**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
23	**fasilitas_tambahan**	longtext	utf8mb4_bin		Yes	*NULL*			Change Drop More
24	**status**	enum('Aktif', 'Nonaktif', 'Dihapus')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
25	**status_acc**	enum('Diterima', 'Ditolak', 'Diajukan')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
26	**tanggal_persetujuan**	datetime			Yes	*NULL*			Change Drop More
27	**alasan**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
28	**created_at**	timestamp			Yes	*NULL*			Change Drop More
29	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.4 Implementasi Tabel properti

b. Tabel foto_properti

Tabel foto_properti digunakan untuk menyimpan data foto-foto yang berkaitan dengan setiap properti yang terdaftar dalam sistem. Tabel ini berfungsi untuk menampilkan dokumentasi visual dari properti sehingga pengguna dapat melihat kondisi aktual bangunan yang ditawarkan. Atribut yang terdapat pada tabel ini terdiri dari foto_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap foto, serta properti_id sebagai *foreign key* yang menghubungkan foto dengan data properti pada tabel properti. Atribut url digunakan untuk menyimpan alamat penyimpanan atau lokasi file

foto pada sistem. Selain itu, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan dan pembaruan data foto secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**foto_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**url**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**created_at**	timestamp			Yes	*NULL*			Change Drop More
5	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.5 Implementasi Tabel foto_properti

c. Tabel video_properti

Tabel video_properti digunakan untuk menyimpan data video yang berkaitan dengan properti yang terdaftar pada sistem. Tabel ini berfungsi untuk menampilkan dokumentasi visual dalam bentuk video agar pengguna dapat memperoleh gambaran kondisi properti secara lebih menyeluruh dan interaktif. Atribut yang terdapat pada tabel ini terdiri dari video_id sebagai primary key yang berfungsi sebagai identitas unik setiap data video, serta properti_id sebagai *foreign key* yang menghubungkan video dengan data pada tabel properti. Atribut url digunakan untuk menyimpan lokasi penyimpanan atau tautan file video yang diunggah ke sistem. Selain itu, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan dan pembaruan data video secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**video_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**url**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**created_at**	timestamp			Yes	*NULL*			Change Drop More
5	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.6 Implementasi Tabel video_properti

d. Tabel properti_aktif

Tabel properti_aktif digunakan untuk menyimpan informasi mengenai properti yang sedang aktif atau telah dipublikasikan sehingga dapat dilihat oleh pengguna umum pada sistem. Tabel ini berfungsi untuk mencatat data properti yang sedang tersedia untuk disewakan atau dijual, beserta periode waktu aktifnya. Atribut yang terdapat pada tabel ini terdiri dari properti_id sebagai *primary key* sekaligus *foreign key* yang menghubungkan data dengan tabel properti, serta pengguna_id sebagai *foreign key* yang mengacu pada pemilik properti. Atribut status digunakan untuk menyimpan kondisi publikasi properti, misalnya "Aktif", "Nonaktif", atau "Selesai". Selain itu, terdapat atribut tanggal_aktif yang digunakan untuk mencatat kapan properti mulai dipublikasikan, serta tanggal_selesai yang menunjukkan kapan periode publikasi berakhir atau properti tersebut sudah tidak tersedia lagi di sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
2	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**status**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**tanggal_aktif**	datetime			Yes	*NULL*			Change Drop More
5	**tanggal_selesai**	datetime			Yes	*NULL*			Change Drop More

Gambar 5.7 Implementasi Tabel properti_aktif

e. Tabel kategori_perubahan

Tabel kategori_perubahan digunakan untuk menyimpan informasi mengenai jenis atau kategori perubahan data yang terjadi pada sistem. Tabel ini berfungsi untuk membantu proses pencatatan dan pelacakan aktivitas perubahan yang dilakukan oleh pengguna terhadap data properti. Atribut yang terdapat pada tabel ini terdiri dari kategori_perubahan_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap kategori perubahan, dan nama_kolom yang digunakan untuk menyimpan nama atau keterangan kolom data yang mengalami perubahan. Selain itu, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data kategori perubahan secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**kategori_perubahan_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**nama_kolom**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
3	**created_at**	timestamp			Yes	*NULL*			Change Drop More
4	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.8 Implementasi Tabel kategorti_perubahan

f. Tabel histori_properti

Tabel histori_properti digunakan untuk menyimpan riwayat perubahan data pada setiap properti yang terdaftar dalam sistem. Tabel ini berfungsi untuk mencatat aktivitas pembaruan atau modifikasi yang dilakukan oleh pengguna atau karyawan terhadap data properti, sehingga setiap perubahan dapat ditelusuri dengan jelas. Atribut yang terdapat pada tabel ini terdiri dari histori_properti_id sebagai primary key yang berfungsi sebagai identitas unik setiap catatan riwayat, serta kategori_perubahan_id, properti_id, dan

karyawan_id yang masing-masing berperan sebagai *foreign key* untuk menghubungkan tabel ini dengan tabel kategori_perubahan, properti, dan karyawan. Atribut tanggal_pengajuan dan tanggal_persetujuan digunakan untuk mencatat waktu pengajuan serta waktu persetujuan perubahan data. Atribut status menyimpan kondisi perubahan dengan nilai enum "Diajukan", "Disetujui", atau "Ditolak". Sementara itu, atribut nilai_sebelum dan nilai_sesudah digunakan untuk menyimpan data lama dan data baru yang mengalami perubahan. Atribut alasan berfungsi untuk mencatat keterangan tambahan yang diberikan pada saat pengajuan perubahan. Terakhir, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan dan pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	histori_properti_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	kategori_perubahan_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	properti_id	bigint(20)		UNSIGNED	No	None			Change Drop More
4	karyawan_id	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
5	tanggal_pengajuan	datetime			Yes	NULL			Change Drop More
6	tanggal_persetujuan	datetime			Yes	NULL			Change Drop More
7	status	enum('Diajukan', 'Disetujui', 'Ditolak')	utf8mb4_unicode_ci		No	Diajukan			Change Drop More
8	nilai_sebelum	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
9	nilai_sesudah	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
10	alasan	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
11	created_at	timestamp			Yes	NULL			Change Drop More
12	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.9 Implementasi Tabel histori_properti

g. Tabel pengajuan_aktivasi

Tabel pengajuan_aktivasi digunakan untuk menyimpan data pengajuan aktivasi ulang terhadap properti yang sebelumnya dinonaktifkan oleh pihak karyawan. Tabel ini berfungsi agar pemilik properti dapat mengajukan

kembali propertinya untuk ditampilkan ke publik setelah melalui proses verifikasi ulang oleh admin atau karyawan yang berwenang. Atribut yang terdapat pada tabel ini terdiri dari pengajuan_aktivasi_id sebagai *primary key* yang menjadi identitas unik setiap pengajuan, serta properti_id dan karyawan_id sebagai *foreign key* yang masing-masing menghubungkan data pengajuan dengan tabel properti dan karyawan. Atribut status digunakan untuk mencatat kondisi pengajuan dengan nilai enum "Diajukan", "Diterima", atau "Ditolak". Atribut alasan menyimpan keterangan tambahan, misalnya alasan pengajuan ulang atau catatan dari karyawan saat proses verifikasi. Selain itu, tanggal_pengajuan digunakan untuk mencatat waktu saat pemilik mengajukan aktivasi kembali, sedangkan tanggal_persetujuan digunakan untuk mencatat waktu keputusan yang diberikan oleh pihak admin atau karyawan. Atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	pengajuan_aktivasi_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	properti_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	karyawan_id	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
4	status	enum('Diajukan', 'Diterima', 'Ditolak')	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
5	alasan	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
6	tanggal_pengajuan	datetime			Yes	NULL			Change Drop More
7	tanggal_persetujuan	datetime			Yes	NULL			Change Drop More
8	created_at	timestamp			Yes	NULL			Change Drop More
9	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.10 Implementasi Tabel karyawan

5.1.3 Implementasi Basis Data Penawaran Harga, Kunjungan, dan Transaksi Properti

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung proses penawaran harga, pengajuan kunjungan, serta transaksi jual beli dan sewa properti dalam sistem informasi properti di kawasan Kota Lama Surabaya. Implementasi basis data pada subbab ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Penawaran Harga, Kunjungan, dan Transaksi Jual Beli/Sewa Properti yang telah dijelaskan pada Bab 4.1.3. Pada subbab ini dijelaskan struktur tabel-tabel yang digunakan untuk mencatat proses negosiasi harga, penjadwalan kunjungan properti, transaksi properti, serta pencatatan pembayaran yang dilakukan oleh pengguna melalui sistem.

a. Tabel tawaran

   Tabel tawaran digunakan untuk menyimpan data penawaran harga yang diajukan oleh calon pembeli atau penyewa terhadap properti yang tersedia di sistem. Tabel ini berfungsi untuk mendukung proses negosiasi antara pemilik properti dan investor, di mana kedua pihak dapat memberikan tawaran serta balasan harga secara interaktif melalui sistem. Atribut yang terdapat pada tabel ini terdiri dari tawaran_id sebagai *primary key* yang menjadi identitas unik setiap data tawaran, properti_id sebagai *foreign key* yang menghubungkan tawaran dengan data pada tabel properti, serta pengguna_id sebagai *foreign key* yang merepresentasikan pengguna yang mengajukan tawaran. Atribut harga_tawaran digunakan untuk menyimpan nilai harga yang

diajukan oleh calon pembeli atau penyewa, sedangkan harga_balasan berisi nilai tawaran balasan dari pemilik properti. Atribut catatan_pengguna dan catatan_pemilik digunakan untuk menyimpan pesan atau keterangan tambahan dari masing-masing pihak selama proses negosiasi. Atribut status menyimpan kondisi penawaran dengan nilai enum "Menunggu", "Menunggu Balasan", "Diterima", atau "Ditolak", sesuai dengan perkembangan proses tawar-menawar. Selain itu, terdapat atribut tanggal_tawaran yang digunakan untuk mencatat waktu pengajuan tawaran, sedangkan *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data tawaran secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**tawaran_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
4	**harga_tawaran**	bigint(20)			No	*None*			Change Drop More
5	**harga_balasan**	bigint(20)			Yes	*NULL*			Change Drop More
6	**catatan_pengguna**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
7	**catatan_pemilik**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
8	**status**	enum('Menunggu','Menunggu Balasan','Diterima',...	utf8mb4_unicode_ci		Yes	Menunggu			Change Drop More
9	**tanggal_tawaran**	timestamp			No	current_timestamp()			Change Drop More
10	**created_at**	timestamp			Yes	*NULL*			Change Drop More
11	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.11 Implementasi Tabel tawaran

b. Tabel kunjungan

Tabel kunjungan digunakan untuk menyimpan data pengajuan jadwal kunjungan yang dilakukan oleh calon penyewa atau pembeli terhadap properti yang tersedia di sistem. Tabel ini berfungsi untuk memfasilitasi proses pertemuan antara calon penyewa dan pemilik properti secara terjadwal, sehingga pemilik dapat menyiapkan waktu dan tempat sesuai kesepakatan. Atribut yang terdapat pada tabel ini terdiri dari kunjungan_id sebagai *primary*

key yang berfungsi sebagai identitas unik setiap data kunjungan, properti_id sebagai *foreign key* yang menghubungkan data kunjungan dengan tabel properti, serta pengguna_id sebagai *foreign key* yang mengacu pada pengguna yang mengajukan kunjungan. Atribut tanggal_dan_jam1, tanggal_dan_jam2, dan tanggal_dan_jam3 digunakan untuk menyimpan tiga opsi jadwal kunjungan yang diajukan oleh investor. Atribut tanggal_dan_jam_disetujui digunakan untuk mencatat waktu disetujui oleh pemilik, sedangkan tanggal_dan_jam_acc berfungsi untuk mencatat jadwal yang dipilih atau disetujui oleh pemilik properti. Atribut status menyimpan kondisi pengajuan kunjungan dengan nilai enum "Menunggu", "Disetujui", atau "Ditolak". Atribut alasan digunakan untuk memberikan keterangan tambahan, misalnya alasan penolakan atau perubahan jadwal kunjungan. Sementara itu, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data kunjungan secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**kunjungan_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
4	**tanggal_dan_jam1**	datetime			Yes	*NULL*			Change Drop More
5	**tanggal_dan_jam2**	datetime			Yes	*NULL*			Change Drop More
6	**tanggal_dan_jam3**	datetime			Yes	*NULL*			Change Drop More
7	**tanggal_dan_jam_disetujui**	datetime			Yes	*NULL*			Change Drop More
8	**tanggal_dan_jam_acc**	datetime			Yes	*NULL*			Change Drop More
9	**status**	enum('Disetujui', 'Ditolak', 'Menunggu')	utf8mb4_unicode_ci		No	Menunggu			Change Drop More
10	**alasan**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
11	**created_at**	timestamp			Yes	*NULL*			Change Drop More
12	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.12 Implementasi Tabel kunjungan

c. Tabel transaksi_properti

Tabel transaksi_properti digunakan untuk menyimpan data transaksi jual beli maupun sewa properti yang dilakukan melalui sistem. Tabel ini berfungsi untuk mencatat seluruh proses transaksi antara pemilik dan pembeli atau penyewa, mulai dari tahap pengajuan hingga status akhir transaksi. Atribut yang terdapat pada tabel ini terdiri dari id sebagai primary key yang menjadi identitas unik setiap transaksi, properti_id, karyawan_id, dan pengguna_id masing-masing sebagai *foreign key* yang menghubungkan data transaksi dengan tabel properti, karyawan, serta pengguna. Atribut ktp_pemilik, alamat_pemilik, nama_pembeli, ktp_pembeli, dan alamat_pembeli digunakan untuk menyimpan informasi identitas pihak yang terlibat dalam transaksi. Atribut total_tagihan berisi total harga atau nilai transaksi yang harus dibayarkan, sedangkan tanggal_transaksi digunakan untuk mencatat waktu transaksi dilakukan dan tanggal_persetujuan untuk mencatat waktu konfirmasi dari pihak pemilik. Selain itu, atribut lama_sewa, tanggal_mulai, dan tanggal_selesai digunakan khusus untuk transaksi sewa, yang mencakup durasi penyewaan properti. Atribut nomor_kontrak dan url_kontrak menyimpan nomor serta tautan file kontrak digital antara kedua belah pihak. Atribut jenis_transaksi memiliki nilai enum “Beli” atau “Sewa” untuk membedakan jenis transaksi, sedangkan status_acc digunakan untuk mencatat status persetujuan transaksi seperti “Disetujui”, “Ditolak”, atau “Menunggu”. Atribut alasan berisi keterangan tambahan yang diberikan saat transaksi ditolak atau direvisi. Atribut metode_pembayaran menyimpan metode pembayaran yang digunakan, misalnya “Penuh” atau “KPR”, sementara

nomor_spk digunakan untuk menyimpan nomor surat perjanjian kerja sama antara kedua belah pihak. Atribut dp digunakan untuk menyimpan nilai uang muka (down payment) apabila pembayaran dilakukan secara bertahap. Terakhir, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan dan pembaruan data transaksi secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**properti_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**karyawan_id**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
4	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
5	**ktp_pemilik**	varchar(16)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
6	**alamat_pemilik**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
7	**nama_pembeli**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
8	**ktp_pembeli**	varchar(16)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
9	**alamat_pembeli**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
10	**total_tagihan**	bigint(20)			Yes	*NULL*			Change Drop More
11	**tanggal_transaksi**	datetime			Yes	*NULL*			Change Drop More
12	**tanggal_persetujuan**	datetime			Yes	*NULL*			Change Drop More
13	**lama_sewa**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
14	**tanggal_mulai**	datetime			Yes	*NULL*			Change Drop More
15	**tanggal_selesai**	datetime			Yes	*NULL*			Change Drop More
16	**nomor_kontrak**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
17	**url_kontrak**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
18	**jenis_transaksi**	enum('Beli', 'Sewa')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
19	**status_acc**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
20	**alasan**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
21	**metode_pembayaran**	enum('Penuh', 'KPR')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
22	**nomor_spk**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
23	**dp**	bigint(20)			Yes	*NULL*			Change Drop More
24	**created_at**	timestamp			Yes	*NULL*			Change Drop More
25	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.13 Implementasi Tabel transaksi_properti

d. Tabel pembayaran_properti

Tabel pembayaran_properti digunakan untuk menyimpan data pembayaran yang dilakukan oleh pembeli atau penyewa terhadap transaksi properti yang telah disetujui. Tabel ini berfungsi untuk mencatat seluruh aktivitas pembayaran, baik pembayaran penuh maupun dp, sehingga proses keuangan

dapat terpantau dengan baik oleh sistem. Atribut yang terdapat pada tabel ini terdiri dari pembayaran_properti_id sebagai *primary key* yang menjadi identitas unik setiap data pembayaran, serta transaksi_properti_id sebagai *foreign key* yang menghubungkan data pembayaran dengan tabel transaksi_properti. Atribut jumlah digunakan untuk menyimpan nilai nominal pembayaran yang dilakukan, sedangkan metode mencatat jenis metode pembayaran yang digunakan, seperti transfer bank. Atribut tanggal_bayar digunakan untuk mencatat waktu pembayaran dilakukan, sedangkan jatuh_tempo menyimpan tanggal batas waktu pembayaran. Atribut status menyimpan kondisi pembayaran seperti "Menunggu Verifikasi", "Lunas", atau "Ditolak". Atribut keterangan digunakan untuk mencatat informasi tambahan mengenai pembayaran, misalnya catatan konfirmasi dari pihak pemilik. Selain itu, url_bukti_pembayaran digunakan untuk menyimpan tautan file bukti pembayaran yang diunggah oleh pengguna. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan dan pembaruan data pembayaran secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	pembayaran_properti_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	transaksi_properti_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	jumlah	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
4	metode	varchar(50)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
5	tanggal_bayar	date			Yes	NULL			Change Drop More
6	jatuh_tempo	date			Yes	NULL			Change Drop More
7	status	varchar(100)	utf8mb4_unicode_ci		Yes				Change Drop More
8	keterangan	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
9	url_bukti_pembayaran	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
10	created_at	timestamp			Yes	NULL			Change Drop More
11	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.14 Implementasi Tabel pembayaran_properti

5.1.4 Implementasi Basis Data Pengelolaan Lokasi Kawasan Kota Lama

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung pengelolaan data lokasi di kawasan Kota Lama Surabaya. Implementasi basis data pada subbab ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Pengelolaan Lokasi Kawasan Kota Lama yang telah dijelaskan pada Bab 4.1.4. Pada subbab ini dijelaskan struktur tabel-tabel yang digunakan untuk menyimpan data lokasi, pengelompokan kategori lokasi, dokumentasi visual lokasi, serta pencatatan riwayat perubahan data lokasi yang dilakukan oleh pengelola lokasi maupun diverifikasi oleh karyawan.

a. Tabel lokasi

   Tabel lokasi digunakan untuk menyimpan data area yang terdaftar dalam sistem. Tabel ini berfungsi untuk mencatat informasi lengkap mengenai lokasi yang dapat dikelola atau disewakan oleh pengguna, termasuk data koordinat, kapasitas, dan status verifikasi lokasi. Atribut yang terdapat pada tabel ini terdiri dari lokasi_id sebagai *primary key* yang menjadi identitas unik setiap lokasi, pengguna_id dan karyawan_id sebagai *foreign key* yang masing-masing menghubungkan data lokasi dengan pengelola dan karyawan yang melakukan verifikasi. Atribut nama_lokasi digunakan untuk menyimpan nama area atau titik lokasi acara, sedangkan alamat_lokasi berfungsi untuk menyimpan alamat lengkap lokasi. Atribut latitude dan longitude digunakan untuk menyimpan koordinat geografis yang memudahkan pemetaan lokasi secara digital. Selain itu, atribut deskripsi berisi penjelasan mengenai kondisi,

fasilitas, atau karakteristik lokasi yang ditawarkan. Atribut nama_kontak_personal dan nomor_kontak_personal digunakan untuk menyimpan data kontak pengelola atau penanggung jawab lokasi. Atribut harga, luas, dan kapasitas digunakan untuk mencatat informasi komersial dan kapasitas area yang tersedia. Atribut jenis_lokasi memiliki nilai enum "Dalam" dan "Luar" untuk membedakan jenis area yang dikelola. Selanjutnya, atribut status digunakan untuk menunjukkan kondisi lokasi dengan nilai "Aktif", "Nonaktif", atau "Dihapus", sedangkan status_acc berfungsi untuk mencatat hasil verifikasi dari pihak admin atau karyawan, dengan nilai "Diterima", "Ditolak", atau "Diajukan". Atribut tanggal_persetujuan digunakan untuk mencatat waktu persetujuan lokasi, dan alasan menyimpan keterangan tambahan apabila lokasi ditolak. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan dan pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**lokasi_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**karyawan_id**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
4	**nama_lokasi**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
5	**alamat_lokasi**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
6	**latitude**	decimal(10,7)			Yes	*NULL*			Change Drop More
7	**longitude**	decimal(10,7)			Yes	*NULL*			Change Drop More
8	**deskripsi**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
9	**nama_kontak_personal**	varchar(45)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
10	**nomor_kontak_personal**	varchar(15)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
11	**harga**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
12	**luas**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
13	**kapasitas**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
14	**jenis_lokasi**	enum('Dalam', 'Luar')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
15	**status**	enum('Aktif', 'Nonaktif', 'Dihapus')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
16	**status_acc**	enum('Diterima', 'Ditolak', 'Diajukan')	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
17	**tanggal_persetujuan**	datetime			Yes	*NULL*			Change Drop More
18	**alasan**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
19	**created_at**	timestamp			Yes	*NULL*			Change Drop More
20	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.15 Implementasi Tabel lokasi

b. Tabel kategori

Tabel kategori digunakan untuk menyimpan data kategori lokasi yang terdapat pada sistem. Tabel ini berfungsi untuk mengelompokkan berbagai jenis lokasi berdasarkan karakteristik atau fungsi tertentu, seperti “Festival Kuliner”, “Pameran Seni”, “Konser Musik”, dan sebagainya. Atribut yang terdapat pada tabel ini terdiri dari kategori_id sebagai *primary key* yang menjadi identitas unik setiap kategori lokasi, serta nama yang digunakan untuk menyimpan nama kategori yang didefinisikan oleh admin sistem. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan dan pembaruan data kategori secara otomatis oleh sistem.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**kategori_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**nama**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
☐	3	**created_at**	timestamp			Yes	*NULL*			Change Drop More
☐	4	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.16 Implementasi Tabel kategori

c. Tabel lokasi_kategori

Tabel lokasi_kategori digunakan untuk menyimpan data relasi antara tabel lokasi dan tabel kategori. Tabel ini berfungsi untuk menghubungkan setiap lokasi dengan kategori tertentu agar sistem dapat menampilkan klasifikasi lokasi sesuai dengan jenisnya, seperti lokasi kuliner, pameran seni, atau konser musik. Atribut yang terdapat pada tabel ini terdiri dari lokasi_kategori_id sebagai *primary key* yang menjadi identitas unik setiap data relasi, lokasi_id sebagai *foreign key* yang menghubungkan data ke tabel lokasi, dan kategori_id sebagai *foreign key* yang menghubungkan data ke tabel

kategori. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**lokasi_kategori_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**lokasi_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**kategori_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
4	**created_at**	timestamp			Yes	*NULL*			Change Drop More
5	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.17 Implementasi Tabel lokasi_kategori

d. Tabel foto_lokasi

Tabel foto_lokasi digunakan untuk menyimpan data foto yang berkaitan dengan lokasi yang terdaftar dalam sistem. Tabel ini berfungsi untuk menampilkan dokumentasi visual dari setiap lokasi, sehingga pengguna dapat melihat kondisi area secara lebih jelas sebelum melakukan kerja sama. Atribut yang terdapat pada tabel ini terdiri dari foto_lokasi_id sebagai *primary key* yang menjadi identitas unik setiap foto lokasi, serta lokasi_id sebagai *foreign key* yang menghubungkan data foto dengan tabel lokasi. Atribut url digunakan untuk menyimpan alamat atau tautan file foto yang diunggah ke sistem. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**foto_lokasi_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**lokasi_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**url**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
4	**created_at**	timestamp			Yes	*NULL*			Change Drop More
5	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.18 Implementasi Tabel foto_lokasi

e. Tabel kategori_perubahan_lokasi

Tabel kategori_perubahan_lokasi digunakan untuk menyimpan data mengenai jenis atau kategori perubahan yang terjadi pada informasi lokasi yang ada di sistem. Tabel ini berfungsi untuk membantu proses pencatatan serta pelacakan pembaruan data lokasi, seperti perubahan nama lokasi, kapasitas. Atribut yang terdapat pada tabel ini terdiri dari kategori_perubahan_lokasi_id sebagai primary key yang berfungsi sebagai identitas unik setiap kategori perubahan lokasi, serta nama_kolom yang digunakan untuk menyimpan nama kolom atau atribut lokasi yang mengalami perubahan. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan dan pembaruan data kategori perubahan lokasi secara otomatis oleh sistem.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**kategori_perubahan_lokasi_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**nama_kolom**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change	Drop	More
☐	3	**created_at**	timestamp			Yes	*NULL*			Change	Drop	More
☐	4	**updated_at**	timestamp			Yes	*NULL*			Change	Drop	More

Gambar 5.19 Implementasi Tabel kategori_perubahan_lokasi

f. Tabel histori_lokasi

Tabel histori_lokasi digunakan untuk menyimpan riwayat perubahan data yang terjadi pada setiap lokasi yang terdaftar di sistem. Tabel ini berfungsi untuk mendokumentasikan aktivitas pembaruan informasi lokasi yang dilakukan oleh pengelola lokasi, sehingga perubahan data dapat dilacak dengan jelas, baik dari sisi waktu maupun jenis perubahannya. Atribut yang terdapat pada tabel ini terdiri dari histori_lokasi_id sebagai *primary key* yang menjadi identitas unik setiap catatan histori, serta kategori_perubahan_lokasi_id, lokasi_id, dan karyawan_id yang

masing-masing berfungsi sebagai foreign key untuk menghubungkan tabel ini dengan tabel kategori_perubahan_lokasi, lokasi, dan karyawan. Atribut tanggal_pengajuan digunakan untuk mencatat waktu pengajuan perubahan data lokasi, sedangkan tanggal_persetujuan berfungsi untuk mencatat waktu ketika perubahan tersebut disetujui oleh pihak yang berwenang. Atribut status menyimpan kondisi proses perubahan dengan nilai enum "Diajukan", "Disetujui", atau "Ditolak". Selain itu, atribut nilai_sebelum dan nilai_sesudah digunakan untuk menyimpan nilai data sebelum dan sesudah perubahan dilakukan. Terakhir, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data histori secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	histori_lokasi_id	bigint(20)			No	None		AUTO_INCREMENT	Change Drop More
2	kategori_perubahan_lokasi_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	lokasi_id	bigint(20)		UNSIGNED	No	None			Change Drop More
4	karyawan_id	bigint(20)		UNSIGNED	Yes	NULL			Change Drop More
5	tanggal_pengajuan	datetime			Yes	NULL			Change Drop More
6	tanggal_persetujuan	datetime			Yes	NULL			Change Drop More
7	status	enum('Diajukan', 'Disetujui', 'Ditolak')	utf8mb4_unicode_ci		No	Diajukan			Change Drop More
8	nilai_sebelum	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
9	nilai_sesudah	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
10	created_at	timestamp			Yes	NULL			Change Drop More
11	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.20 Implementasi Tabel histori_lokasi

5.1.5 Implementasi Basis Data Penyewaan Lokasi Acara

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung proses penyewaan lokasi acara dalam sistem informasi properti di kawasan Kota Lama Surabaya. Implementasi basis data pada subbab ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Penyewaan Lokasi Acara

yang telah dijelaskan pada Bab 4.1.5. Pada subbab ini dijelaskan struktur tabel-tabel yang digunakan untuk menyimpan data transaksi acara, dokumentasi kegiatan, serta pencatatan pembayaran penyewaan lokasi acara yang dilakukan oleh penyelenggara acara melalui sistem.

a. Tabel transaksi_acara

Tabel transaksi_acara digunakan untuk menyimpan data kegiatan atau acara yang diselenggarakan pada lokasi yang terdaftar di sistem. Tabel ini berfungsi untuk mencatat seluruh informasi terkait penyelenggaraan acara, termasuk data penyewa, lokasi acara, biaya sewa, serta status verifikasi acara oleh pihak pengelola lokasi. Atribut yang terdapat pada tabel ini terdiri dari transaksi_acara_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap data acara, serta lokasi_id, pengguna_id, kategori_id sebagai *foreign key* yang masing-masing menghubungkan data acara dengan tabel lokasi, pengguna, dan kategori. Atribut nama_acara digunakan untuk menyimpan nama kegiatan, sedangkan deskripsi berisi keterangan tambahan mengenai tema atau rincian acara yang akan diadakan. Atribut status_acc dan status masing-masing digunakan untuk mencatat hasil verifikasi dari pihak pengelola serta status pelaksanaan acara, seperti “Menunggu”, “Disetujui”, atau “Selesai”. Atribut tanggal_mulai, tanggal_selesai, dan tanggal_persetujuan digunakan untuk mencatat jadwal kegiatan serta waktu persetujuan dari pihak pengelola lokasi. Atribut harga menyimpan biaya sewa lokasi untuk pelaksanaan acara, sedangkan nama_bank dan nomor_rek digunakan untuk

mencatat informasi rekening pembayaran pengelola lokasi. Selain itu, atribut nama_kontak_personal dan nomor_kontak_personal menyimpan informasi kontak penanggung jawab acara. Atribut jumlah_peserta digunakan untuk mencatat kapasitas peserta yang mengikuti kegiatan, sedangkan url_proposal, nomor_kontrak, dan url_kontrak menyimpan dokumen pendukung kegiatan seperti proposal dan surat perjanjian kerja sama. Atribut alasan berfungsi untuk mencatat keterangan tambahan apabila terjadi penolakan atau pembatalan acara. Atribut open_tenant dan jumlah_tenant digunakan untuk mencatat informasi jumlah tenant yang berpartisipasi dalam acara. Atribut metode_pembayaran berisi jenis pembayaran yang digunakan, misalnya "Penuh" atau "Cicilan", sedangkan dp menyimpan jumlah uang muka yang telah dibayarkan oleh penyewa lokasi. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data acara secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	transaksi_acara_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	lokasi_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	pengguna_id	bigint(20)		UNSIGNED	No	None			Change Drop More
4	kategori_id	bigint(20)		UNSIGNED	No	None			Change Drop More
5	nama_acara	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
6	deskripsi	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
7	status_acc	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
8	status	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
9	tanggal_mulai	datetime			Yes	NULL			Change Drop More
10	tanggal_selesai	datetime			Yes	NULL			Change Drop More
11	tanggal_persetujuan	datetime			Yes	NULL			Change Drop More
12	harga	bigint(20)			Yes	NULL			Change Drop More
13	nama_bank	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
14	nomor_rek	varchar(20)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
15	nama_kontak_personal	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
16	nomor_kontak_personal	varchar(15)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
17	jumlah_peserta	int(11)			Yes	NULL			Change Drop More
18	url_proposal	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
19	nomor_kontrak	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
20	url_kontrak	varchar(255)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
21	alasan	text	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
22	open_tenant	tinyint(1)			No	0			Change Drop More
23	jumlah_tenant	int(11)			No	0			Change Drop More
24	metode_pembayaran	enum('Penuh', 'Cicilan')	utf8mb4_unicode_ci		No	None			Change Drop More
25	dp	bigint(20)			No	None			Change Drop More
26	created_at	timestamp			Yes	NULL			Change Drop More
27	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.21 Implementasi Tabel transaksi_acara

b. Tabel foto_acara

Tabel foto_acara digunakan untuk menyimpan dokumentasi foto yang berkaitan dengan kegiatan atau acara yang diselenggarakan pada lokasi yang terdaftar di sistem. Tabel ini berfungsi untuk menampilkan kegiatan serta mendukung kebutuhan publikasi yang akan dilaksanakan oleh penyewa lokasi atau penyelenggara acara. Atribut yang terdapat pada tabel ini terdiri dari foto_acara_id sebagai *primary key* yang menjadi identitas unik setiap foto acara, serta acara_id sebagai *foreign key* yang menghubungkan foto dengan data acara pada tabel transaksi_acara. Atribut url digunakan untuk menyimpan alamat atau tautan file foto dokumentasi yang diunggah ke sistem. Selain itu,

atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan dan pembaruan data foto secara otomatis oleh sistem.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**foto_acara_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**acara_id**	bigint(20)		UNSIGNED	No	*None*			Change	Drop	More
☐	3	**url**	text	utf8mb4_unicode_ci		Yes	*NULL*			Change	Drop	More
☐	4	**created_at**	timestamp			Yes	*NULL*			Change	Drop	More
☐	5	**updated_at**	timestamp			Yes	*NULL*			Change	Drop	More

Gambar 5.22 Implementasi Tabel foto_acara

c. Tabel pembayaran_acara

Tabel pembayaran_acara digunakan untuk menyimpan data pembayaran yang dilakukan oleh penyelenggara acara atau pihak penyewa lokasi terhadap transaksi kegiatan yang disetujui oleh pengelola lokasi. Tabel ini berfungsi untuk mencatat seluruh aktivitas pembayaran acara agar proses administrasi keuangan dapat dilakukan secara transparan dan terdokumentasi di dalam sistem. Atribut yang terdapat pada tabel ini terdiri dari pembayaran_acara_id sebagai *primary key* yang menjadi identitas unik setiap data pembayaran, serta transaksi_acara_id sebagai *foreign key* yang menghubungkan data pembayaran dengan tabel transaksi_acara. Atribut jumlah digunakan untuk menyimpan nilai nominal pembayaran yang dilakukan oleh penyelenggara acara, sedangkan metode menyimpan informasi metode pembayaran seperti transfer. Atribut tanggal_bayar digunakan untuk mencatat waktu pembayaran dilakukan, sedangkan jatuh_tempo digunakan untuk menyimpan batas waktu pembayaran apabila sistem menerapkan skema cicilan. Atribut status menyimpan kondisi pembayaran seperti “Menunggu Verifikasi”, “Lunas”,

atau “Ditolak”. Atribut keterangan digunakan untuk mencatat informasi tambahan terkait proses pembayaran, dan url_bukti_pembayaran digunakan untuk menyimpan tautan bukti pembayaran yang diunggah ke sistem. Terakhir, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data pembayaran secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**pembayaran_acara_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**transaksi_acara_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**jumlah**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
4	**metode**	varchar(50)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
5	**tanggal_bayar**	date			Yes	*NULL*			Change Drop More
6	**jatuh_tempo**	date			Yes	*NULL*			Change Drop More
7	**status**	varchar(100)	utf8mb4_unicode_ci		No				Change Drop More
8	**keterangan**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
9	**url_bukti_pembayaran**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
10	**created_at**	timestamp			Yes	*NULL*			Change Drop More
11	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.23 Implementasi Tabel pembayaran_acara

5.1.6 Implementasi Basis Data Penyewaan Tenant Acara

Subbab ini menjelaskan implementasi basis data yang digunakan untuk mendukung proses penyewaan tenant pada acara yang diselenggarakan di kawasan Kota Lama Surabaya. Implementasi basis data pada subbab ini mengacu pada hasil perancangan Entity Relationship Diagram (ERD) Penyewaan Tenant Acara yang telah dijelaskan pada Bab 4.1.6. Pada subbab ini dijelaskan struktur tabel-tabel yang digunakan untuk mengelola data lokasi tenant, kategori tenant, pendaftaran tenant, serta pencatatan pembayaran tenant yang dilakukan selama pelaksanaan acara.

a. Tabel tenant_lokasi

Tabel tenant_lokasi digunakan untuk menyimpan data tenant yang menempati lokasi tertentu pada acara yang diselenggarakan di kawasan Kota Lama Surabaya. Tabel ini berfungsi untuk mengatur penyewaan tempat bagi tenant pada suatu acara, termasuk informasi posisi tenant, harga sewa, serta status pembayaran dan ketersediaan tempat. Atribut yang terdapat pada tabel ini terdiri dari tenant_lokasi_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap data tenant lokasi, serta transaksi_acara_id sebagai *foreign key* yang menghubungkan tenant dengan tabel transaksi_acara tempat acara berlangsung. Atribut latitude dan longitude digunakan untuk menyimpan koordinat posisi tenant di area acara, sehingga penempatan tenant dapat dipetakan secara akurat. Atribut nama_tenant digunakan untuk menyimpan nama tenant yang menempati lokasi tersebut, sedangkan harga digunakan untuk mencatat nilai biaya sewa tempat. Atribut status_pembayaran memiliki nilai enum “Belum Dibayar”, “DP”, dan “Lunas” untuk mencatat kondisi pembayaran tenant terhadap penyelenggara acara. Atribut status digunakan untuk menunjukkan kondisi ketersediaan tenant, seperti “Tersedia”, “Dipesan”, atau “Tidak Tersedia”. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan dan pembaruan data secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	tenant_lokasi_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	transaksi_acara_id	bigint(20)		UNSIGNED	No	None			Change Drop More
3	latitude	decimal(10,7)			No	None			Change Drop More
4	longitude	decimal(10,7)			No	None			Change Drop More
5	nama_tenant	varchar(100)	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
6	harga	int(11)			Yes	NULL			Change Drop More
7	status_pembayaran	enum('Belum Dibayar', 'DP', 'Lunas')	utf8mb4_unicode_ci		Yes	NULL			Change Drop More
8	status	enum('Tersedia', 'Dipesan', 'Tidak Tersedia')	utf8mb4_unicode_ci		Yes	Tersedia			Change Drop More
9	created_at	timestamp			Yes	NULL			Change Drop More
10	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.24 Implementasi Tabel tennat_lokasi

b. Tabel kategori_tenant

Tabel kategori_tenant digunakan untuk menyimpan data kategori tenant dalam kegiatan atau acara di kawasan Kota Lama Surabaya. Tabel ini berfungsi untuk mengelompokkan tenant berdasarkan jenis usaha atau layanan yang mereka tawarkan, seperti “Kuliner”, “Kerajinan”, “Souvenir”, atau “Jasa”. Atribut yang terdapat pada tabel ini terdiri dari kategori_tenant_id sebagai *primary key* yang berfungsi sebagai identitas unik setiap kategori tenant, serta nama_kategori yang digunakan untuk menyimpan nama atau jenis kategori tenant yang dikelola oleh admin. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data kategori tenant secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	kategori_tenant_id	bigint(20)		UNSIGNED	No	None		AUTO_INCREMENT	Change Drop More
2	nama_kategori	varchar(100)	utf8_general_ci		No	None			Change Drop More
3	created_at	timestamp			Yes	NULL			Change Drop More
4	updated_at	timestamp			Yes	NULL			Change Drop More

Gambar 5.25 Implementasi Tabel kategori_tenant

c. Tabel kategori_tenant_lokasi

Tabel kategori_tenant_lokasi digunakan untuk menyimpan data relasi antara kategori tenant dengan lokasi tempat tenant tersebut. Tabel ini berfungsi untuk menghubungkan jenis usaha tenant dengan area atau lokasi yang ditempati, sehingga sistem dapat menampilkan informasi kategori tenant yang ada di setiap lokasi secara lebih terstruktur. Atribut yang terdapat pada tabel ini terdiri dari id sebagai *primary key* yang menjadi identitas unik setiap data relasi, tenant_lokasi_id sebagai *foreign key* yang menghubungkan data dengan tabel tenant_lokasi, serta kategori_tenant_id sebagai *foreign key* yang menghubungkan data dengan tabel kategori_tenant. Selain itu, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data secara otomatis oleh sistem.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action		
☐	1	**id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change	Drop	More
☐	2	**tenant_lokasi_id**	bigint(20)		UNSIGNED	No	*None*			Change	Drop	More
☐	3	**kategori_tenant_id**	bigint(20)		UNSIGNED	No	*None*			Change	Drop	More
☐	4	**created_at**	timestamp			Yes	*NULL*			Change	Drop	More
☐	5	**updated_at**	timestamp			Yes	*NULL*			Change	Drop	More

Gambar 5.26 Implementasi Tabel kategori_tenant_lokasi

d. Tabel tenant_daftar

Tabel tenant_daftar digunakan untuk menyimpan data pendaftaran yang menyewa tenant dalam acara yang diselenggarakan pada kawasan Kota Lama Surabaya.Tabel ini berfungsi untuk mencatat informasi lengkap mengenai tenant, termasuk identitas, kategori usaha, status pembayaran, serta dokumen pendukung yang diperlukan untuk proses verifikasi dan kerja sama. Atribut yang terdapat pada tabel ini terdiri dari tenant_daftar_id sebagai *primary key*

yang menjadi identitas unik setiap pendaftaran tenant, tenant_lokasi_id sebagai *foreign key* yang menghubungkan tenant dengan lokasi yang ditempati, serta pengguna_id sebagai *foreign key* yang menghubungkan tenant dengan data pengguna yang melakukan pendaftaran. Atribut nama_tenant digunakan untuk menyimpan nama usaha atau tenant yang didaftarkan, sedangkan kategori digunakan untuk mencatat jenis usaha yang dijalankan. Atribut no_ktp digunakan untuk menyimpan nomor identitas pemilik tenant sebagai data validasi. Atribut deskripsi digunakan untuk menjelaskan secara singkat profil atau bidang usaha tenant. Selain itu, atribut status digunakan untuk mencatat kondisi pendaftaran tenant seperti "Menunggu Konfirmasi", "Diterima", atau "Ditolak". Atribut status_pembayaran memiliki nilai enum "Belum Bayar", "DP", dan "Lunas" untuk mencatat status pembayaran tenant terhadap penyelenggara acara. Atribut jumlah_dp dan total_bayar digunakan untuk menyimpan data nilai uang muka dan total pembayaran. Atribut nama_bank serta nomor_rek digunakan untuk menyimpan informasi rekening tenant yang digunakan dalam transaksi keuangan. Atribut file_pendukung berisi berkas tambahan seperti surat izin usaha atau dokumen promosi, sedangkan url_kontrak digunakan untuk menyimpan tautan dokumen kontrak kerja sama tenant. Terakhir, atribut *created_at* dan *updated_at* digunakan untuk mencatat waktu pembuatan serta pembaruan data tenant secara otomatis oleh sistem.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	**tenant_daftar_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
2	**tenant_lokasi_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
3	**pengguna_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
4	**nama_tenant**	varchar(255)	utf8_general_ci		No	*None*			Change Drop More
5	**kategori**	varchar(255)	utf8_general_ci		No	*None*			Change Drop More
6	**no_ktp**	varchar(255)	utf8_general_ci		Yes	*NULL*			Change Drop More
7	**deskripsi**	text	utf8_general_ci		Yes	*NULL*			Change Drop More
8	**status**	varchar(100)	utf8_general_ci		No	Menunggu Konfirmasi			Change Drop More
9	**status_pembayaran**	enum('Belum Bayar', 'DP', 'Lunas')	utf8_general_ci		Yes	*NULL*			Change Drop More
10	**jumlah_dp**	bigint(20)			Yes	*NULL*			Change Drop More
11	**total_bayar**	bigint(20)			Yes	*NULL*			Change Drop More
12	**nama_bank**	varchar(100)	utf8_general_ci		Yes	*NULL*			Change Drop More
13	**nomor_rek**	varchar(20)	utf8_general_ci		Yes	*NULL*			Change Drop More
14	**file_pendukung**	varchar(255)	utf8_general_ci		Yes	*NULL*			Change Drop More
15	**url_kontrak**	varchar(255)	utf8_general_ci		Yes	*NULL*			Change Drop More
16	**created_at**	timestamp			Yes	*NULL*			Change Drop More
17	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.27 Implementasi Tabel tenant_daftar

e. Tabel pembayaran_tenant

Tabel pembayaran_tenant digunakan untuk menyimpan data pembayaran yang dilakukan oleh tenant yang telah terdaftar dan berpartisipasi pada acara di kawasan Kota Lama Surabaya. Tabel ini berfungsi untuk mencatat setiap transaksi pembayaran yang dilakukan oleh tenant kepada pihak penyelenggara acara, baik pembayaran uang muka maupun pelunasan, sehingga seluruh aktivitas keuangan tenant dapat terdokumentasi dengan baik. Atribut yang terdapat pada tabel ini terdiri dari pembayaran_tenant_id sebagai *primary key* yang menjadi identitas unik setiap data pembayaran, serta tenant_daftar_id sebagai *foreign key* yang menghubungkan data pembayaran dengan tabel tenant_daftar. Atribut jumlah digunakan untuk menyimpan nilai nominal pembayaran yang dilakukan oleh tenant, sedangkan metode digunakan untuk mencatat jenis metode pembayaran yang digunakan, seperti transfer. Atribut tanggal_bayar digunakan untuk mencatat waktu transaksi dilakukan, sementara jatuh_tempo digunakan untuk menyimpan batas waktu

pembayaran. Atribut status digunakan untuk menunjukkan kondisi pembayaran seperti “Menunggu Verifikasi”, “Lunas”, atau “Ditolak”. Atribut keterangan berfungsi untuk mencatat catatan tambahan terkait pembayaran, misalnya alasan penolakan atau konfirmasi dari pihak penyelenggara acara. Selain itu, atribut url_bukti_pembayaran digunakan untuk menyimpan tautan bukti pembayaran yang diunggah ke sistem. Terakhir, atribut *created_at* dan *updated_at* berfungsi untuk mencatat waktu pembuatan serta pembaruan data pembayaran secara otomatis oleh sistem.

	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	**pembayaran_tenant_id**	bigint(20)		UNSIGNED	No	*None*		AUTO_INCREMENT	Change Drop More
☐	2	**tenant_daftar_id**	bigint(20)		UNSIGNED	No	*None*			Change Drop More
☐	3	**jumlah**	bigint(20)		UNSIGNED	Yes	*NULL*			Change Drop More
☐	4	**metode**	varchar(50)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
☐	5	**tanggal_bayar**	date			Yes	*NULL*			Change Drop More
☐	6	**jatuh_tempo**	date			Yes	*NULL*			Change Drop More
☐	7	**status**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
☐	8	**keterangan**	varchar(100)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
☐	9	**url_bukti_pembayaran**	varchar(255)	utf8mb4_unicode_ci		Yes	*NULL*			Change Drop More
☐	10	**created_at**	timestamp			Yes	*NULL*			Change Drop More
☐	11	**updated_at**	timestamp			Yes	*NULL*			Change Drop More

Gambar 5.28 Implementasi Tabel pembayaran_tenant

5.2 Implementasi Kode Program

Pada sub bab ini akan dijelaskan implementasi dari desain proses yang telah dibuat pada bab sebelumnya. Setiap penjelasan dilengkapi dengan potongan kode untuk menggambarkan cara kerja program secara lebih jelas. Bagian ini berisi potongan kode dari sistem tugas akhir. Implementasi dilakukan untuk menggambarkan bagaimana sistem berjalan dalam mengelola data properti, lokasi, acara, tenant, serta transaksi yang terjadi di dalamnya.

5.2.1 Implementasi Fitur Pengelolaan Akun Pengguna

Pada sub bab ini akan dijelaskan implementasi dari desain proses yang telah dibuat pada bab sebelumnya. Setiap penjelasan dilengkapi dengan potongan kode untuk menggambarkan cara kerja program secara lebih jelas. Bagian ini berisi potongan kode dari sistem yang menangani proses manajemen akun pengguna, mulai dari penambahan akun baru hingga proses aktivasi akun yang dilakukan oleh atau karyawan Dinas Pariwisata.

a. Penambahan Akun Pengguna

Proses penambahan akun pengguna dilakukan melalui halaman registrasi pada sistem. Fitur ini memungkinkan calon pengguna, seperti Investor, Pemilik Properti, dan Pengelola Lokasi, untuk membuat akun agar dapat mengakses sistem sesuai dengan peran masing-masing. Pada proses ini, sistem melakukan validasi terhadap seluruh input yang dikirimkan melalui form pendaftaran. Data yang harus diisi meliputi nama, email, nomor handphone, peran pengguna (role), serta kata sandi. Fungsi validate() digunakan untuk memastikan seluruh data wajib diisi dengan benar dan email belum pernah digunakan sebelumnya pada tabel pengguna. Setelah data dinyatakan valid, sistem akan menentukan status awal akun berdasarkan peran pengguna. Untuk pengguna dengan peran Investor, akun akan langsung berstatus Aktif, sedangkan untuk pengguna dengan peran Pemilik Properti dan Pengelola Lokasi, akun akan disimpan dengan status Menunggu Verifikasi hingga diperiksa oleh pihak karyawan. Selanjutnya, data pengguna akan disimpan ke dalam tabel pengguna, dan kata sandi yang dimasukkan akan dienkripsi menggunakan fungsi Hash::make()

agar keamanan data tetap terjaga. Potongan kode berikut menunjukkan implementasi proses penambahan akun pengguna.

Listing 5.1 Potongan Kode Penambahan Akun Pengguna

```
public function daftar(Request $request)
    {
        $request->validate([
            'nama' => 'required|string|max:255',
            'email' => 'required|email|unique:pengguna,email',
            'no_hp' => 'required|string|max:20',
            'role' => 'required|in:Investor,Pemilik Properti,Pengelola Lokasi',
            'password' => 'required|min:6',
        ]);

        $verifikasi = false;
        $status = 'menunggu';

        if ($request->role === 'Investor') {
            $status = 'Aktif';
        }

        $pengguna = Pengguna::create([
            'nama' => $request->nama,
            'email' => $request->email,
            'no_hp' => $request->no_hp,
            'role' => $request->role,
            'password' => Hash::make($request->password),
            'status' => $status,
        ]);

        return response()->json([
            'message' => 'Registrasi berhasil',
            'data' => $pengguna
        ], 201);
    }
```

b. Aktivasi Akun oleh Karyawan Dinas Pariwisata

Proses aktivasi akun dilakukan oleh karyawan Dinas Pariwisata sebagai pihak yang berwenang dalam melakukan verifikasi dan persetujuan akun pengguna baru.

Fitur ini berfungsi untuk memastikan bahwa setiap pengguna yang mendaftar, terutama dengan peran Pemilik Properti dan Pengelola Lokasi, telah melalui tahap validasi data sebelum diizinkan mengakses fitur-fitur utama sistem. Pada proses ini, sistem akan mencari data pengguna berdasarkan ID pengguna yang dikirimkan melalui pendaftaran. Jika data pengguna ditemukan, maka status akun akan diperbarui menjadi Aktif dan sistem akan mencatat tanggal persetujuan menggunakan Carbon::now() untuk menyimpan waktu aktivasi secara otomatis. Namun, jika pengguna tidak ditemukan di basis data, sistem akan menampilkan pesan kesalahan (error response) dengan status 404. Potongan kode berikut menunjukkan implementasi proses aktivasi akun oleh karyawan dinas.

Listing 5.2 Potongan Kode Aktivasi Akun oleh Karyawan Disbudporapar

```
public function aktifkanPengguna($id)
{
    $pengguna = Pengguna::find($id);
    if (!$pengguna) {
        return response()->json([
            'success' => false,
            'message' => 'Pengguna tidak ditemukan'
        ], 404);
    }
    $pengguna->status = 'Aktif';
    $pengguna->tanggal_persetujuan = Carbon::now();
    $pengguna->save();
    return response()->json([
        'success' => true,
        'message' => 'Status pengguna berhasil diubah menjadi aktif',
        'data' => $pengguna
    ]);
}
```

5.2.2 Implementasi Manajemen Data Properti

Pada subbab ini akan dijelaskan implementasi dari desain proses manajemen data properti yang telah dibuat pada bab sebelumnya. Setiap penjelasan dilengkapi dengan potongan kode untuk menggambarkan cara kerja program secara lebih jelas. Bagian ini membahas proses pengelolaan data properti yang meliputi penambahan data properti, verifikasi oleh karyawan Dinas Pariwisata, pengajuan dan persetujuan perubahan data, penonaktifan, serta pengaktifan kembali properti yang dikelola di dalam sistem.

a. Penambahan Data Properti Baru

Proses penambahan data properti dilakukan oleh pemilik properti melalui formulir input yang memuat informasi identitas, karakteristik fisik, lokasi, serta media pendukung (foto dan video). Pada tahap ini sistem melakukan validasi terhadap seluruh kolom wajib seperti pengguna_id, nama_properti, jenis_komersial, harga, dimensi bangunan/tanah, dan alamat lengkap. Validasi memanfaatkan Request::validate() untuk memastikan tipe data sesuai, nilai berada pada rentang yang diperbolehkan, serta relasi foreign key (exists:pengguna,pengguna_id) terpenuhi. Data yang telah lolos validasi kemudian disimpan ke tabel properti. Atribut fasilitas_tambahan—yang dikirim sebagai array dari form—diubah lebih dahulu menjadi JSON agar konsisten tersimpan di basis data. Status awal properti diatur menjadi Diajukan melalui kolom status_acc sebagai penanda bahwa entri tersebut menunggu proses verifikasi karyawan sebelum dapat dipublikasikan. Fitur ini juga mendukung unggah multi-foto dan satu video pendukung. Berkas media disimpan ke disk public menggunakan store(), lalu path hasil unggah direkam pada tabel

foto_properti dan video_properti melalui relasi Eloquent ($properti->fotoProperti() dan $properti->videoProperti()). Di akhir proses, respons JSON diberikan beserta eager loaded relasi media untuk memudahkan penampilan di antarmuka.

Listing 5.3 Potongan Kode Penambahan Data Properti Baru

```
public function store(Request $request)
   {
      $validatedData = $request->validate([
         'pengguna_id' => 'required|integer|exists:pengguna,pengguna_id',
         'nama_properti' => 'required|string|max:255',
         'jenis_komersial' => 'required|in:Jual,Sewa',
         'nama_pemilik' => 'required|string|max:255',
         'harga' => 'required|numeric',
         'nomor_shm' => 'nullable|string',
         'luas_bangunan' => 'required|numeric',
         'luas_tanah' => 'required|numeric',
         'deskripsi' => 'nullable|string',
         'alamat_lengkap' => 'required|string',
         'latitude' => 'nullable|numeric',
         'longitude' => 'nullable|numeric',
         'jumlah_ruangan' => 'nullable|string',
         'tahun_pendirian' => 'nullable|string',
         'renovasi_terakhir' => 'nullable|string',
         'kondisi_bangunan' => 'nullable|string',
          'gaya_arsitekstur' => 'nullable|string',
         'listrik' => 'nullable|string',
         'air_pam' => 'nullable|string',
         'parkir' => 'nullable|string',
         'fasilitas_tambahan' => 'nullable|array',
         'foto.*' => 'nullable|image|mimes:jpeg,png,jpg|max:2048',
         'video' => 'nullable|file|mimes:mp4,mov,avi,wmv|max:10240',
      ]);

      $properti = Properti::create([
         ...$validatedData,
         'fasilitas_tambahan' => $request->fasilitas_tambahan
            ? json_encode($request->fasilitas_tambahan)
            : null,
```

Listing 5.3 Potongan Kode Penambahan Data Properti Baru (Lanjutan)

```
        'status_acc' => 'Diajukan',
    ]);

    if ($request->hasFile('foto')) {
        foreach ($request->file('foto') as $file) {
            $path = $file->store('foto_properti', 'public');
            $properti->fotoProperti()->create([
                'url' => 'storage/' . $path,
            ]);
        }
    }
    if ($request->hasFile('video')) {
        $videoPath = $request->file('video')->store('video_properti', 'public');
        $properti->videoProperti()->create([
            'url' => 'storage/' . $videoPath,
        ]);
    }
    return response()->json([
        'success' => true,
        'message' => 'Data properti berhasil disimpan.',
        'data' => $properti->load('fotoProperti', 'videoProperti'),
    ], 201);
}
```

b. Verifikasi Penambahan Data Properti Baru

Proses verifikasi dilakukan oleh karyawan Dinas Pariwisata setelah pemilik properti mengajukan data properti pada tahap penambahan. Tujuan dari proses ini adalah untuk menentukan status akhir pengajuan melalui atribut status_acc dengan pilihan Diterima, Ditolak, atau Diajukan. Sebelum melakukan perubahan data, sistem terlebih dahulu melakukan validasi terhadap input yang diberikan, seperti status pengajuan, alasan (jika ada), serta karyawan_id yang harus valid pada tabel karyawan. Apabila pengajuan Diterima, maka data properti akan diubah menjadi aktif dengan memperbarui atribut status menjadi Aktif, menandai karyawan_id sebagai petugas yang melakukan verifikasi, dan mencatat waktu persetujuan melalui atribut

tanggal_persetujuan. Selain itu, properti juga akan dipublikasikan melalui pencatatan ke tabel properti_aktif dengan masa aktif selama tiga bulan sejak tanggal persetujuan, yang diatur melalui atribut tanggal_aktif dan tanggal_selesai. Metode updateOrCreate() digunakan agar sistem dapat memperbarui data properti aktif tanpa menimbulkan duplikasi. Sebaliknya, apabila pengajuan Ditolak, sistem akan mengubah status properti menjadi Nonaktif, mencatat alasan penolakan, menandai petugas pemeriksa melalui karyawan_id, serta menyimpan waktu persetujuan pada tanggal_persetujuan. Sementara itu, status Diajukan dipertahankan untuk data properti yang masih menunggu proses verifikasi. Mekanisme ini memastikan seluruh hasil verifikasi tercatat secara konsisten pada tabel properti serta tersinkronisasi dengan tabel properti_aktif pada saat properti diterima dan dipublikasikan ke sistem.

Listing 5.4 Potongan Kode Verifikasi Penambahan Data Properti Baru

```
public function verifikasi($id, Request $request)
    {
        $properti = Properti::find($id);
        if (!$properti) {
            return response()->json([
                'success' => false,
                'message' => 'Data properti tidak ditemukan.'
            ], 404);
        }

        $request->validate([
            'status_acc' => 'required|in:Diterima,Ditolak,Diajukan',
            'alasan' => 'nullable|string|max:255',
            'karyawan_id' => 'nullable|integer|exists:karyawan,karyawan_id',
        ]);
        $properti = Properti::find($id);
```

Listing 5.4 Potongan Kode Verifikasi Penambahan Data Properti Baru

(Lanjutan)

```
    if (!$properti) {
        return response()->json([
            'success' => false,
            'message' => 'Data properti tidak ditemukan.'
        ], 404);
    }
    $request->validate([
        'status_acc' => 'required|in:Diterima,Ditolak,Diajukan',
        'alasan' => 'nullable|string|max:255',
        'karyawan_id' => 'nullable|integer|exists:karyawan,karyawan_id',
    ]);
    $properti->status_acc = $request->status_acc;
    if ($request->status_acc === 'Diterima') {
        $properti->status = 'Aktif';
        $properti->karyawan_id = $request->karyawan_id;
        $properti->alasan = null;
        $properti->tanggal_persetujuan = Carbon::now();
        $tanggalAktif = Carbon::now();
        $tanggalSelesai = $tanggalAktif->copy()->addMonths(3);
        PropertiAktif::updateOrCreate(
            [
                'properti_id' => $properti->properti_id,
                'pengguna_id' => $properti->pengguna_id, // pastikan kolom ini ada di
tabel properti
            ],
            [
                'status' => 'Aktif',
                'tanggal_aktif' => $tanggalAktif,
                'tanggal_selesai' => $tanggalSelesai,
            ]
        );
    } elseif ($request->status_acc === 'Ditolak') {
        $properti->status = 'Nonaktif';
        $properti->alasan = $request->alasan ?? '-';
        $properti->karyawan_id = $request->karyawan_id;
        $properti->tanggal_persetujuan = Carbon::now();
    }
```

Listing 5.4 Potongan Kode Verifikasi Penambahan Data Properti Baru (Lanjutan)

```
    $properti->save();
    return response()->json([
        'success' => true,
        'message' => 'Status verifikasi properti berhasil diperbarui.',
        'data' => $properti
    ], 200);
}
```

c. Pengajuan Perubahan Data Properti

Fitur ini memungkinkan pemilik properti mengajukan pembaruan atas beberapa atribut properti sekaligus (misalnya harga, deskripsi, fasilitas, dan lain-lain) tanpa langsung mengubah data utama. Setiap usulan perubahan dicatat ke tabel histori_properti sebagai trail audit dengan status awal Diajukan, sehingga karyawan dapat meninjau dan memutuskan untuk menyetujui atau menolak pada proses verifikasi. Pada tahap ini, sistem memvalidasi pasangan kolom dan nilai yang dikirim, memetakan setiap nama kolom ke kategori_perubahan (melalui nama_kolom), mengambil nilai_sebelum dari data properti saat ini, lalu menyimpan nilai_sesudah ke catatan histori. Untuk kolom bertipe media (foto/video) yang dikirim sebagai array/JSON, sistem menormalkan menjadi string JSON agar seragam tersimpan di basis data. Output berupa daftar entri histori yang berhasil dibuat sebagai bukti pengajuan.

Listing 5.5 Potongan Kode Pengajuan Perubahan Data Properti

```
public function ubahProperti(Request $request, $propertiId)
 {
    $properti = Properti::findOrFail($propertiId);
```

Listing 5.5 Potongan Kode Pengajuan Perubahan Data Properti (Lanjutan)

```
    $kolomList = $request->input('kolom', []);
    $nilaiList = $request->input('nilai', []);

    if (empty($kolomList) || count($kolomList) !== count($nilaiList)) {
        return response()->json([
            'success' => false,
            'message' => 'Data yang dikirim tidak valid atau tidak lengkap.'
        ], 400);
    }

    $historiAll = [];

    foreach ($kolomList as $index => $namaKolom) {
        $nilaiSesudah = $nilaiList[$index];

        $kategori = KategoriPerubahan::where('nama_kolom',
$namaKolom)->first();
        if (!$kategori)
            continue;

        $nilaiSebelum = $properti->{$namaKolom};

        if (in_array($namaKolom, ['foto', 'video'])) {
         if (is_string($nilaiSesudah)) {
                $decoded = json_decode($nilaiSesudah, true);
                $nilaiSesudah = $decoded ?? $nilaiSesudah;
            }
            $nilaiSebelum = $nilaiSebelum ? json_encode($nilaiSebelum) : null;
            $nilaiSesudah = is_array($nilaiSesudah) ? json_encode($nilaiSesudah) :
$nilaiSesudah;
        }

        $histori = HistoriProperti::create([
            'properti_id' => $properti->properti_id,
            'karyawan_id' => null,
            'kategori_perubahan_id' => $kategori->kategori_perubahan_id,
            'nilai_sebelum' => $nilaiSebelum,
            'nilai_sesudah' => $nilaiSesudah,
            'alasan' => null,
            'tanggal_pengajuan' => now(),
            'status' => 'diajukan',
```

Listing 5.5 Potongan Kode Pengajuan Perubahan Data Properti (Lanjutan)

```
        ]);

        $historiAll[] = $histori;
    }

    return response()->json([
        'success' => true,
        'message' => 'Semua perubahan properti berhasil diajukan.',
        'histori' => $historiAll,
    ]);
}
```

d. Verifikasi Pengajuan Perubahan Data Properti

Proses verifikasi pengajuan perubahan data properti dilakukan oleh karyawan Dinas Pariwisata untuk meninjau dan menyetujui seluruh pembaruan yang diajukan oleh pemilik. Fitur ini berfungsi untuk memastikan bahwa setiap perubahan data properti yang diusulkan telah diperiksa kebenaran dan kesesuaiannya sebelum disetujui dan diterapkan ke dalam tabel utama properti. Pada tahap ini, sistem akan terlebih dahulu memastikan bahwa ID karyawan pengaju verifikasi telah dikirimkan dan valid. Kemudian, sistem mencari seluruh data perubahan dari tabel histori_properti dengan status Diajukan untuk properti tertentu. Jika tidak ditemukan data pengajuan, sistem akan mengembalikan pesan bahwa tidak ada perubahan yang perlu diverifikasi. Apabila data pengajuan ditemukan, setiap histori akan diperbarui menjadi Disetujui dengan mencatat waktu persetujuan (tanggal_persetujuan) dan karyawan_id dari petugas yang melakukan verifikasi. Selanjutnya, sistem akan mengambil data dari tabel kategori_perubahan untuk menentukan kolom mana yang perlu diperbarui pada tabel properti. Jika kolom tersebut ada di struktur tabel, maka

nilai pada atribut nilai_sesudah dari histori akan diterapkan ke tabel properti, menggantikan data sebelumnya. Khusus untuk kolom dengan tipe data khusus seperti fasilitas_tambahan, sistem menyesuaikan pembaruan agar nilai tetap konsisten dalam format JSON. Dengan cara ini, seluruh perubahan yang telah disetujui oleh karyawan secara otomatis memperbarui data utama properti dan menjaga integritas histori perubahan yang terdokumentasi.

Listing 5.6 Potongan Kode Verifikasi Pengajuan Perubahan Data Properti

```
public function setujuiSemua(Request $request, $properti_id)
   {
      $karyawan_id = $request->karyawan_id;

      if (!$karyawan_id) {
         return response()->json([
            'success' => false,
            'message' => 'ID karyawan wajib dikirim.'
         ], 400);
      }
   $historiList = HistoriProperti::where('properti_id', $properti_id)
         ->where('status', 'Diajukan')
         ->get();

      if ($historiList->isEmpty()) {
         return response()->json([
            'success' => false,
            'message' => 'Tidak ada perubahan yang diajukan untuk properti ini.'
         ], 404);
      }

      $properti = Properti::find($properti_id);
      if (!$properti) {
         return response()->json([
            'success' => false,
            'message' => 'Data properti tidak ditemukan.'
         ], 404);
```

Listing 5.6 Potongan Kode Verifikasi Pengajuan Perubahan Data Properti (Lanjutan)

```
        }
        foreach ($historiList as $histori) {
            $histori->status = 'Disetujui';
            $histori->tanggal_persetujuan = now();
            $histori->karyawan_id = $karyawan_id;
            $histori->save();

            $kategori = KategoriPerubahan::find($histori->kategori_perubahan_id);
            if ($kategori) {
                $kolom = $kategori->nama_kolom;

                if (Schema::hasColumn('properti', $kolom)) {
                    $properti->$kolom = $histori->nilai_sesudah;
                } elseif ($kolom === 'fasilitas_tambahan') {
                    $properti->fasilitas_tambahan = $histori->nilai_sesudah;
                }
            }
        }
        $properti->save();
        return response()->json([
          'success' => true,
           'message' => 'Semua perubahan berhasil disetujui oleh karyawan dan data
properti telah diperbarui.',
           'data' => [
                'properti' => $properti,
                'disetujui_oleh' => $karyawan_id
            ]
        ], 200);
    }
```

e. Penonaktifan Properti

Fitur penonaktifan properti digunakan oleh karyawan untuk menghentikan sementara publikasi suatu properti dari halaman publik. Proses ini diawali dengan validasi masukan berupa alasan penonaktifan dan karyawan_id sebagai penanda petugas yang melakukan tindakan. Sistem kemudian memastikan keberadaan data pada tabel properti_aktif untuk properti terkait serta memuat data induknya pada tabel

properti. Apabila data tidak ditemukan, sistem mengembalikan respons gagal yang sesuai. Setelah data valid, sistem memperbarui status properti menjadi Nonaktif dan menyimpan alasan penonaktifan pada tabel properti. Status pada tabel properti_aktif turut diubah menjadi Nonaktif agar properti tidak lagi muncul pada tampilan publik. Untuk menjaga jejak audit, perubahan dicatat ke tabel histori_properti dalam dua entri terpisah: satu entri untuk perubahan status dari nilai sebelumnya ke Nonaktif, dan satu entri untuk pembaruan alasan penonaktifan. Masing-masing entri histori menyertakan karyawan_id, waktu tanggal_pengajuan dan tanggal_persetujuan (diisi saat yang sama karena tindakan langsung disahkan), serta status histori diset ke Disetujui. Dengan demikian, setiap tindakan nonaktif terdokumentasi lengkap, konsisten antara tabel operasional (properti dan properti_aktif) dan tabel audit (histori_properti).

Listing 5.7 Potongan Kode Penonaktifkan Properti

```
public function nonaktifkanProperti(Request $request, $propertiId)
    {
        $request->validate([
            'alasan' => 'required|string|max:500',
            'karyawan_id' => 'required|integer',
        ]);
        $propertiAktif = PropertiAktif::where('properti_id', $propertiId)->first();
        if (!$propertiAktif) {
            return response()->json([
                'success' => false,
                'message' => 'Properti aktif tidak ditemukan.'
            ], 404);
        }

        $properti = Properti::find($propertiId);
        if (!$properti) {
            return response()->json([
```

Listing 5.7 Potongan Kode Penonaktifkan Properti (Lanjutan)

```
            'success' => false,
            'message' => 'Data properti tidak ditemukan.'
        ], 404);
    }
    $nilaiSebelum = [
        'status' => $properti->status,
        'alasan' => $properti->alasan,
    ];
    $properti->status = 'Nonaktif';
    $properti->alasan = $request->alasan;
    $properti->save();
$propertiAktif->status = 'Nonaktif';
    $propertiAktif->save();
    $kategoriStatus = KategoriPerubahan::where('nama_kolom', 'status')->first();
    $kategoriAlasan = KategoriPerubahan::where('nama_kolom', 'alasan')->first();
    if ($kategoriStatus) {
        HistoriProperti::create([
            'properti_id' => $propertiId,
            'kategori_perubahan_id' => $kategoriStatus->kategori_perubahan_id,
            'karyawan_id' => $request->karyawan_id,
            'nilai_sebelum' => $nilaiSebelum['status'],
            'nilai_sesudah' => 'Nonaktif',
            'tanggal_pengajuan' => now(),
            'tanggal_persetujuan' => now(),
            'status' => 'Disetujui',
        ]);
    }
    if ($kategoriAlasan) {
        HistoriProperti::create([
            'properti_id' => $propertiId,
            'kategori_perubahan_id' => $kategoriAlasan->kategori_perubahan_id,
            'karyawan_id' => $request->karyawan_id,
            'nilai_sebelum' => $nilaiSebelum['alasan'],
            'nilai_sesudah' => $request->alasan,
            'tanggal_pengajuan' => now(),
            'tanggal_persetujuan' => now(),
            'status' => 'Disetujui',
        ]);
    }
    return response()->json([
        'success' => true,
```

Listing 5.7 Potongan Kode Penonaktifkan Properti (Lanjutan)

```
        'message' => 'Properti berhasil dinonaktifkan dan seluruh perubahan dicatat
di histori_properti.'
    ]);
}
```

f. Pengaktifan Kembali Properti

Fitur aktivasi kembali properti digunakan oleh karyawan Dinas Pariwisata untuk memproses pengajuan reaktivasi dari pemilik yang ingin mempublikasikan ulang properti yang sebelumnya dinonaktifkan. Proses ini penting untuk memastikan bahwa setiap properti yang kembali aktif telah melalui tahap pemeriksaan dan persetujuan resmi oleh pihak berwenang. Sebelum menjalankan proses, sistem akan memvalidasi keberadaan properti_id yang dikirim oleh pengguna. Jika ID tidak dikirim atau data properti tidak ditemukan, sistem akan menampilkan pesan kesalahan yang sesuai. Setelah validasi berhasil, sistem memulai transaksi database (DB::beginTransaction()). Selanjutnya, sistem mengambil kategori perubahan untuk kolom status dan alasan dari tabel kategori_perubahan. Jika kategori tersebut ditemukan, maka sistem membuat dua catatan baru pada tabel histori_properti — satu untuk mencatat perubahan status dari Nonaktif menjadi Aktif, dan satu lagi untuk menghapus alasan penonaktifan sebelumnya. Kedua histori tersebut langsung diset dengan status Disetujui dan mencatat waktu pengajuan serta persetujuan secara otomatis menggunakan now(). Setelah pencatatan histori, status properti diperbarui menjadi Aktif dan nilai alasan diset menjadi null. Tabel properti_aktif juga diperbarui agar properti kembali tampil di halaman publik. Jika terdapat data pada tabel

pengajuan_aktivasi, sistem memperbarui statusnya menjadi Diterima, menambahkan karyawan_id yang melakukan verifikasi, serta mencatat waktu persetujuan pada kolom tanggal_persetujuan. Semua proses ini dijalankan di dalam satu transaksi agar integritas data tetap terjaga. Jika seluruh langkah berhasil, transaksi akan disimpan (DB::commit()), sedangkan jika terjadi kesalahan, sistem akan melakukan pembatalan otomatis (DB::rollback()) dan menampilkan pesan error.

Listing 5.8 Potongan Kode Pengaktifan Kembali Properti

```
public function activateAgain(Request $request)
  {
    if (!$request->properti_id) {
      return response()->json([
        'success' => false,
        'message' => 'Properti ID wajib dikirim.'
      ], 400);
    }

    \DB::beginTransaction();
    try {
      $properti = Properti::find($request->properti_id);
      if (!$properti) {
        return response()->json([
          'success' => false,
          'message' => 'Properti tidak ditemukan.'
        ], 404);
      }
      $statusKategori = \DB::table('kategori_perubahan')
        ->where('nama_kolom', 'status')
        ->first();
      $alasanKategori = \DB::table('kategori_perubahan')
        ->where('nama_kolom', 'alasan')
        ->first();
      if (!$statusKategori || !$alasanKategori) {
        return response()->json([
          'success' => false,
          'message' => 'Kategori untuk status atau alasan tidak ditemukan.'
        ], 500);
      }
```

Listing 5.8 Potongan Kode Pengaktifan Kembali Properti (Lanjutan)

```
            HistoriProperti::create([
                'properti_id' => $properti->properti_id,
                'kategori_perubahan_id' => $statusKategori->kategori_perubahan_id,
                'nilai_sebelum' => $properti->status,
                'nilai_sesudah' => 'Aktif',
                'status' => 'Disetujui',
                'tanggal_pengajuan' => now(),
                'tanggal_persetujuan' => now()
            ]);
            HistoriProperti::create([
                'properti_id' => $properti->properti_id,
                'kategori_perubahan_id' => $alasanKategori->kategori_perubahan_id,
                'nilai_sebelum' => $properti->alasan,
                'nilai_sesudah' => null,
                'status' => 'Disetujui',
                'tanggal_pengajuan' => now(),
                'tanggal_persetujuan' => now()
            ]);
            $properti->status = 'Aktif';
            $properti->alasan = null;
            $properti->save();
            PropertiAktif::where('properti_id', $properti->properti_id)
                ->update(['status' => 'Aktif']);
            $pengajuan = PengajuanAktivasi::where('properti_id',
$properti->properti_id)
                ->orderBy('tanggal_pengajuan', 'desc')
                ->first();
            if ($pengajuan) {
                $pengajuan->status = 'Diterima';
                $pengajuan->karyawan_id = $request->karyawan_id;
                $pengajuan->tanggal_persetujuan = now();
                $pengajuan->save();
            }
            \DB::commit();
            return response()->json([
                'success' => true,
                'message' => 'Properti berhasil diaktifkan kembali.',
                'data' => $properti
            ]);
        } catch (\Exception $e) {
            \DB::rollback();
```

Listing 5.8 Potongan Kode Pengaktifan Kembali Properti (Lanjutan)

```
        return response()->json([
            'success' => false,
            'message' => 'Terjadi kesalahan: ' . $e->getMessage()
        ], 500);
    }
}
```

5.2.3 Implementasi Penawaran Harga, Kunjungan, dan Transaksi Properti

Pada subbab ini akan dijelaskan implementasi dari desain proses penawaran harga, kunjungan, dan transaksi properti yang telah dirancang pada bab sebelumnya. Setiap proses dijelaskan dengan potongan kode untuk memperjelas alur kerja sistem. Pembahasan meliputi pengajuan dan persetujuan tawaran harga, pengajuan serta penjadwalan kunjungan properti, pencatatan transaksi jual beli maupun sewa, pengelolaan pembayaran, serta pembuatan kontrak properti secara otomatis di dalam sistem.

a. Pengajuan Tawaran Harga Properti

Fitur ini memungkinkan pengguna (calon pembeli/penyewa) mengajukan tawaran harga terhadap properti yang dipilih. Sistem terlebih dahulu melakukan validasi terhadap data yang dikirim, mencakup keberadaan relasi properti dan pengguna (exists), nilai harga_tawaran yang harus numerik dan lebih dari nol, serta catatan_pengguna yang bersifat opsional. Jika valid, sistem membuat entri baru pada tabel tawaran dengan status awal Menunggu sebagai penanda proses negosiasi belum ditindaklanjuti pemilik. Respons dikembalikan dalam format JSON.

Listing 5.9 Potongan Kode Pengajuan Tawaran Harga Properti

```
public function store(Request $request)
   {
      $request->validate([
         'properti_id' => 'required|exists:properti,properti_id',
         'pengguna_id' => 'required|exists:pengguna,pengguna_id',
         'harga_tawaran' => 'required|numeric|min:1',
         'catatan_pengguna' => 'nullable|string',
      ]);
      $tawaran = Tawaran::create([
         'properti_id' => $request->properti_id,
         'pengguna_id' => $request->pengguna_id,
         'harga_tawaran' => $request->harga_tawaran,
         'catatan_pengguna' => $request->catatan_pengguna,
         'status' => 'Menunggu',
      ]);
      return response()->json([
         'message' => 'Tawaran berhasil dikirim.',
         'data' => $tawaran
      ], 201);
   }
```

b. Penerimaan Tawaran Harga Properti

Fitur ini digunakan oleh pemilik properti untuk menyetujui tawaran harga yang diajukan oleh calon pembeli atau penyewa. Proses dimulai dengan mencari data tawaran berdasarkan ID tawaran serta data properti yang terkait melalui properti_id. Setelah data ditemukan, sistem membandingkan nilai harga lama dari properti dengan harga baru yang diajukan dalam tawaran. Jika pemilik menyetujui tawaran tersebut, maka status pada tabel tawaran akan diperbarui menjadi Diterima, dan sistem secara otomatis mengubah nilai harga properti sesuai dengan harga baru yang telah disepakati. Selain memperbarui harga, sistem juga mencatat perubahan ini ke dalam tabel histori_properti untuk menjaga rekam jejak transaksi dan memastikan bahwa setiap perubahan signifikan pada data properti terdokumentasi dengan baik. Histori

mencakup informasi kategori perubahan (diambil dari tabel kategori_perubahan dengan kolom nama_kolom = 'harga'), nilai harga sebelum dan sesudah perubahan, tanggal pengajuan tawaran (tanggal_pengajuan), serta tanggal persetujuan (tanggal_persetujuan). Dengan pencatatan ini, setiap penyesuaian harga akibat negosiasi dapat dilacak secara transparan oleh pihak pemilik.

Listing 5.10 Potongan Kode Penerimaan Tawaran Harga Properti

```
public function terima($id)
   {
      $tawaran = Tawaran::findOrFail($id);
      $properti = Properti::findOrFail($tawaran->properti_id);

      $hargaLama = $properti->harga;
      $hargaBaru = $tawaran->harga_tawaran;

      $tawaran->update(['status' => 'Diterima']);
      $properti->update(['harga' => $hargaBaru]);
      $kategori = KategoriPerubahan::where('nama_kolom', 'harga')->first();
      HistoriProperti::create([
         'kategori_perubahan_id' => $kategori ? $kategori->kategori_perubahan_id :
null,
         'properti_id' => $properti->properti_id,
         'tanggal_pengajuan' => $tawaran->created_at,
         'tanggal_persetujuan' => Carbon::now(),
         'status' => 'Disetujui',
         'nilai_sebelum' => $hargaLama,
         'nilai_sesudah' => $hargaBaru,
      ]);
      return response()->json(['message' => 'Tawaran diterima dan harga properti
telah diperbarui.']);
   }
```

c. Pengajuan Kunjungan Properti

Fitur ini memungkinkan calon penyewa/pembeli mengajukan jadwal kunjungan ke properti dengan menyediakan hingga tiga opsi tanggal dan waktu. Sistem terlebih dahulu melakukan validasi terhadap identitas properti dan pengguna

(melalui aturan exists) serta format tanggal–waktu dengan pola Y-m-d H:i:s. Apabila validasi gagal, sistem mengembalikan respons 422 berisi rincian kesalahan untuk diperbaiki oleh pengguna. Jika seluruh data valid, sistem membentuk payload kunjunganData berisi properti_id, pengguna_id, dan hingga tiga alternatif tanggal_dan_jam lalu membuat entri baru pada tabel kunjungan. Respons dikembalikan dalam format JSON.

Listing 5.11 Potongan Kode Pengajuan Kunjungan Properti

```
public function store(Request $request)
  {
    $validator = Validator::make($request->all(), [
      'properti_id' => 'required|exists:properti,properti_id',
      'pengguna_id' => 'required|exists:pengguna,pengguna_id',
      'tanggal_dan_jam1' => 'nullable|date_format:Y-m-d H:i:s',
      'tanggal_dan_jam2' => 'nullable|date_format:Y-m-d H:i:s',
      'tanggal_dan_jam3' => 'nullable|date_format:Y-m-d H:i:s',
    ]);
    if ($validator->fails()) {
      return response()->json([
        'success' => false,
        'message' => 'Validasi gagal',
        'errors' => $validator->errors()
      ], 422);
    }
    $kunjunganData = [
      'properti_id' => $request->properti_id,
      'pengguna_id' => $request->pengguna_id,
      'tanggal_dan_jam1' => $request->tanggal_dan_jam1,
      'tanggal_dan_jam2' => $request->tanggal_dan_jam2,
      'tanggal_dan_jam3' => $request->tanggal_dan_jam3,
    ];
    $kunjungan = Kunjungan::create($kunjunganData);
    return response()->json([
      'success' => true,
      'message' => 'Kunjungan berhasil diajukan.',
      'data' => $kunjungan
    ]);}
```

d. Penerimaan Kunjungan Properti

Fitur ini digunakan oleh pemilik properti untuk menyetujui salah satu jadwal kunjungan yang diajukan oleh calon penyewa atau pembeli. Proses ini memastikan bahwa interaksi antara pengguna dan pemilik berjalan terjadwal dan terdokumentasi dengan baik di sistem. Sebelum menyetujui jadwal, sistem akan melakukan validasi terhadap data yang dikirim oleh pemilik, yaitu ID kunjungan dan tanggal_dan_jam_acc yang harus berupa format tanggal yang valid. Jika validasi gagal atau data kunjungan tidak ditemukan, sistem akan memberikan respons dengan pesan kesalahan yang sesuai. Apabila data valid, sistem akan memperbarui data kunjungan dengan mengisi kolom tanggal_dan_jam_acc sebagai waktu kunjungan yang telah disetujui dan mencatat tanggal_dan_jam_disetujui dengan waktu saat ini menggunakan now(). Selanjutnya, kolom status akan diubah menjadi Disetujui untuk menandakan bahwa permintaan kunjungan telah diterima dan dikonfirmasi oleh pemilik properti. Melalui fitur ini, sistem dapat memastikan bahwa seluruh jadwal kunjungan yang diterima tercatat secara akurat, sehingga memudahkan koordinasi antara pihak calon pembeli/penyewa dan pemilik properti.

Listing 5.12 Potongan Kode Penerimaan Kunjungan Properti

```
public function terimaKunjungan(Request $request)
   {
      $request->validate([
         'kunjungan_id' => 'required|integer',
         'tanggal_dan_jam_acc' => 'required|date',
      ]);
      $kunjungan = Kunjungan::find($request->kunjungan_id);
      if (!$kunjungan) {
         return response()->json([
            'success' => false,
            'message' => 'Kunjungan tidak ditemukan'
         ]);
```

```
        }
        $kunjungan->tanggal_dan_jam_acc = $request->tanggal_dan_jam_acc;
        $kunjungan->tanggal_dan_jam_disetujui = now();
        $kunjungan->status = 'Disetujui';
        $kunjungan->save();
        return response()->json([
            'success' => true,
            'message' => 'Kunjungan disetujui'
        ]);
    }
```

e. Pengajuan Transaksi Properti

Fitur ini digunakan untuk mencatat transaksi jual beli maupun sewa atas suatu properti. Sistem terlebih dahulu melakukan validasi terhadap data yang dikirim, meliputi identitas properti dan pengguna, jenis transaksi (Beli/Sewa), identitas pihak pembeli/penyewa, serta metode pembayaran (Penuh/KPR). Setelah validasi berhasil, sistem mengambil harga terkini dari tabel properti untuk diisikan sebagai total_tagihan pada entri transaksi. Untuk metode pembayaran KPR, sistem menyediakan inisialisasi nilai DP—menggunakan input pengguna jika tersedia, atau default sebesar 20% dari harga properti. Sementara untuk jenis transaksi Sewa, sistem juga menampung lama_sewa sesuai permintaan. Status awal transaksi dicatat sebagai Menunggu Konfirmasi, dan tanggal_transaksi diisi waktu saat ini, sehingga proses verifikasi selanjutnya oleh karyawan dapat dilacak dengan jelas.

Listing 5.13 Potongan Kode Pengajuan Transaksi Properti

```
public function store(Request $request)
    {
        $request->validate([
```

Listing 5.13 Potongan Kode Pengajuan Transaksi Properti (Lanjutan)

```
'properti_id' => 'required|exists:properti,properti_id',
        'pengguna_id' => 'required|integer',
        'jenis_transaksi' => 'required|string|in:Beli,Sewa',
        'nama_pembeli' => 'required|string',
        'ktp_pembeli' => 'required|string',
        'alamat_pembeli' => 'required|string',
        'metode_pembayaran' => 'required|string|in:Penuh,KPR',
        'dp' => 'nullable|numeric',
    ]);
    $properti = Properti::find($request->properti_id);
    if (!$properti) {
        return response()->json([
            'success' => false,
            'message' => 'Properti tidak ditemukan.',
        ], 404);
    }
    $transaksi = new TransaksiProperti();
    $transaksi->properti_id = $request->properti_id;
    $transaksi->pengguna_id = $request->pengguna_id;
    $transaksi->jenis_transaksi = $request->jenis_transaksi;
    $transaksi->total_tagihan = $properti->harga;
    $transaksi->nama_pembeli = $request->nama_pembeli;
    $transaksi->ktp_pembeli = $request->ktp_pembeli;
    $transaksi->alamat_pembeli = $request->alamat_pembeli;
    $transaksi->tanggal_transaksi = Carbon::now();
    $transaksi->status_acc = 'Menunggu Konfirmasi';
    $transaksi->metode_pembayaran = $request->metode_pembayaran;
    if ($request->metode_pembayaran === 'KPR') {
        $transaksi->dp = $request->dp ?? round($properti->harga * 0.2);
    }
    if ($request->jenis_transaksi === 'Sewa') {
        $transaksi->lama_sewa = $request->lama_sewa;
    }
    $transaksi->save();
    return response()->json([
        'success' => true,
        'message' => 'Transaksi properti berhasil disimpan.',
        'data' => $transaksi,
    ], 201);
}
```

f. Pembuatan Kontrak Properti

Fitur Pembuatan Kontrak Properti digunakan untuk menghasilkan dokumen kontrak secara otomatis berdasarkan data transaksi properti yang tersimpan di dalam sistem. Proses ini bertujuan untuk memastikan kontrak jual beli atau sewa properti dapat dibuat secara terstruktur dan terdokumentasi dengan baik. Proses dimulai ketika sistem menerima permintaan pembuatan kontrak berdasarkan ID transaksi properti. Sistem kemudian mengambil data transaksi beserta data terkait, seperti data properti, pemilik properti, dan pembayaran. Selanjutnya, sistem menentukan jenis transaksi, apakah Sewa atau Jual Beli, untuk memilih template kontrak yang sesuai. Sebelum kontrak dibuat, sistem melakukan pengecekan ketersediaan template kontrak. Jika template tidak ditemukan, sistem akan memberikan respons kesalahan. Apabila template tersedia, sistem akan membentuk dokumen kontrak dalam format PDF dengan memuat informasi transaksi dan tanggal pembuatan kontrak.Dokumen kontrak yang dihasilkan kemudian disimpan ke dalam sistem, dan data transaksi akan diperbarui dengan nomor kontrak serta URL kontrak. Sebagai tahap akhir, sistem menyediakan file kontrak untuk diunduh oleh pengguna. Dengan adanya fitur ini, proses pembuatan kontrak properti dapat dilakukan secara otomatis, efisien, dan terdokumentasi dengan baik.

Listing 5.14 Potongan Kode Pembuatan Kontrak Properti

```
public function generateKontrak($id)
    {
        $transaksi = TransaksiProperti::with(['properti',
'pembayaranProperti'])->findOrFail($id);
```

Listing 5.14 Potongan Kode Pembuatan Kontrak Properti (Lanjutan)

```
        $jenis = $transaksi->jenis_transaksi ?? 'Jual Beli';
        $view = $jenis === 'Sewa' ? 'kontrak_sewa' : 'kontrak_jual_beli';
        if (!view()->exists($view)) {
            return response()->json(['error' => "Template kontrak untuk $jenis tidak
ditemukan."], 404);
        }
        $data = [
            'transaksi' => $transaksi,
            'properti' => $transaksi->properti,
            'pemilik' => $transaksi->properti->pemilik ?? null,
            'pembayaran' => $transaksi->pembayaranProperti,
            'jenis' => $jenis,
            'tanggal' => now()->translatedFormat('d F Y'),
        ];

        $pdf = PDF::loadView($view, $data)
            ->setPaper('a4', 'portrait')
            ->setOptions([
                'isHtml5ParserEnabled' => true,
                'isRemoteEnabled' => true,
                'defaultFont' => 'DejaVu Sans',
            ]);

        $folder = storage_path('app/public/kontrak');
        if (!file_exists($folder)) {
            mkdir($folder, 0755, true);
        }

        $filename = 'Kontrak-' . $jenis . '-' . $transaksi->id . '.pdf';
        $pdfPath = $folder . '/' . $filename;
        $pdf->save($pdfPath);

        $transaksi->nomor_kontrak = 'KTR-' . $transaksi->id;
        $transaksi->url_kontrak = url('storage/kontrak/' . $filename);
        $transaksi->save();

        return $pdf->download($filename);
    }
```

g. Pembayaran Transaksi Properti

Fitur Pembayaran Transaksi Properti digunakan untuk mengelola perubahan status pembayaran yang dilakukan oleh pengguna berdasarkan hasil verifikasi. Proses ini memastikan bahwa pembayaran yang dilakukan tercatat dengan status yang sesuai di dalam sistem. Proses dimulai ketika sistem menerima permintaan perubahan status pembayaran berdasarkan ID pembayaran properti. Sistem akan melakukan pengecekan untuk memastikan data pembayaran tersedia. Jika data pembayaran tidak ditemukan, sistem akan memberikan respons kesalahan.

Selanjutnya, sistem melakukan validasi terhadap status yang dikirim, yaitu Setujui atau Tolak. Apabila status yang diberikan adalah Setujui, sistem akan membandingkan tanggal pembayaran dengan tanggal jatuh tempo. Jika pembayaran dilakukan melewati tanggal jatuh tempo, maka status pembayaran akan diubah menjadi Terlambat, sedangkan jika tidak, status akan diubah menjadi Bayar. Jika status yang diberikan adalah Tolak, sistem akan mengubah status pembayaran menjadi Upload Ulang Pembayaran dan menghapus data bukti pembayaran yang sebelumnya diunggah. Setelah status diperbarui, sistem menyimpan perubahan data pembayaran dan memberikan respons bahwa proses perubahan status pembayaran telah berhasil dilakukan. Melalui fitur ini, sistem dapat memastikan pengelolaan pembayaran transaksi properti berjalan secara terkontrol dan terdokumentasi dengan baik.

Listing 5.15 Potongan Kode Pembayaran Transaksi Properti

```
public function ubahStatus(Request $request, $id)
  {
    $pembayaran = PembayaranProperti::find($id);
```

Listing 5.15 Potongan Kode Pembayaran Transaksi Properti (Lanjutan)

```
    if (!$pembayaran) {
        return response()->json([
            'success' => false,
            'message' => 'Pembayaran tidak ditemukan.'
        ], 404);
    }
    $validated = $request->validate([
        'status' => 'required|string|in:Setujui,Tolak'
    ]);

    $tanggalBayar = $pembayaran->tanggal_bayar;
    $tanggalJatuhTempo = $pembayaran->tanggal_jatuh_tempo;

    if ($validated['status'] === 'Setujui') {
        if ($tanggalBayar && $tanggalJatuhTempo) {
            if (strtotime($tanggalBayar) > strtotime($tanggalJatuhTempo)) {
                $pembayaran->status = 'Terlambat';
            } else {
                $pembayaran->status = 'Bayar';
            }
        } else {
            $pembayaran->status = 'Bayar';
        }
    } elseif ($validated['status'] === 'Tolak') {
        $pembayaran->status = 'Upload Ulang Pembayaran';
        $pembayaran->url_bukti_pembayaran = null;
    }

    $pembayaran->save();

    return response()->json([
        'success' => true,
        'message' => 'Status pembayaran berhasil diperbarui.',
        'data' => $pembayaran
    ]);
}
```

5.2.4 Implementasi Manajemen Lokasi Kawasan Kota Lama Surabaya

Pada subbab ini akan dijelaskan implementasi dari desain proses manajemen lokasi pada Kawasan Kota Lama Surabaya yang telah disusun pada bab sebelumnya. Penjelasan disertai dengan potongan kode untuk menggambarkan proses kerja sistem secara jelas. Bagian ini mencakup proses penambahan data lokasi oleh pengelola lokasi, verifikasi data lokasi oleh karyawan Dinas Pariwisata, serta pengajuan perubahan data lokasi yang dikelola secara terstruktur di dalam sistem.

a. Penambahan Data Lokasi Baru

Fitur Penambahan Data Lokasi Baru digunakan oleh pengelola lokasi untuk menambahkan informasi lokasi yang akan disediakan dalam sistem. Proses ini bertujuan untuk menyimpan data lokasi secara lengkap. Proses dimulai ketika pengelola lokasi mengirimkan data lokasi melalui sistem. Sistem akan melakukan validasi terhadap data yang diterima, seperti identitas pengelola lokasi, informasi lokasi, koordinat geografis, kategori lokasi, serta file foto yang diunggah. Apabila validasi gagal, data tidak akan diproses lebih lanjut.

Setelah data dinyatakan valid, sistem akan menyimpan data lokasi ke dalam basis data dengan status Diajukan. Selanjutnya, jika pengguna memilih kategori lokasi, sistem akan menyimpan relasi kategori tersebut. Sistem juga akan menyimpan foto lokasi yang diunggah ke dalam penyimpanan dan mencatat alamat file foto pada basis data. Setelah seluruh data berhasil disimpan, sistem akan mengembalikan respons bahwa data lokasi berhasil ditambahkan beserta informasi lokasi yang telah

tersimpan. Melalui fitur ini, sistem dapat memastikan bahwa data lokasi tercatat secara rapi dan siap untuk proses verifikasi selanjutnya.

Listing 5.16 Potongan Kode Penambahan Data Lokasi Baru

```
public function store(Request $request)
   {
      $request->validate([
         'pengguna_id' => 'required|integer|exists:pengguna,pengguna_id',
         'nama_lokasi' => 'nullable|string|max:255',
         'alamat_lokasi' => 'nullable|string',
         'latitude' => 'nullable|numeric',
         'longitude' => 'nullable|numeric',
         'deskripsi' => 'nullable|string',
         'nama_kontak_personal' => 'nullable|string|max:45',
         'nomor_kontak_personal' => 'nullable|string|max:15',
         'harga' => 'nullable|numeric',
         'luas' => 'nullable|string|max:100',
         'kapasitas' => 'nullable|string|max:100',
         'jenis_lokasi' => 'nullable|in:Dalam,Luar',
         'kategori' => 'array',
         'foto.*' => 'image|mimes:jpeg,png,jpg|max:2048',
      ]);
      $lokasi = Lokasi::create([
         'pengguna_id' => $request->pengguna_id,
         'nama_lokasi' => $request->nama_lokasi,
         'alamat_lokasi' => $request->alamat_lokasi,
         'latitude' => $request->latitude,
         'longitude' => $request->longitude,
         'deskripsi' => $request->deskripsi,
         'nama_kontak_personal' => $request->nama_kontak_personal,
         'nomor_kontak_personal' => $request->nomor_kontak_personal,
         'harga' => $request->harga,
         'luas' => $request->luas,
         'kapasitas' => $request->kapasitas,
         'jenis_lokasi' => $request->jenis_lokasi,
         'status_acc' => 'Diajukan',
      ]);
      if ($request->has('kategori') && is_array($request->kategori)) {
         $lokasi->kategori()->attach($request->kategori);
      }
      if ($request->hasFile('foto')) {
         foreach ($request->file('foto') as $file) {
```

Listing 5.16 Potongan Kode Penambahan Data Lokasi Baru (Lanjutan)

```
            $path = $file->store('foto_lokasi', 'public');
            FotoLokasi::create([
                'lokasi_id' => $lokasi->lokasi_id,
                'url' => '/storage/' . $path,
            ]);
        }
    }

    $lokasi->load(['kategori', 'fotoLokasi']);

    return response()->json([
        'success' => true,
        'message' => 'Data lokasi berhasil disimpan.',
        'data' => $lokasi->load('kategori')
    ], 201);
}
```

b. Verifikasi Penambahan Data Lokasi Baru

Fitur Verifikasi Penambahan Data Lokasi Baru digunakan oleh karyawan untuk melakukan pemeriksaan dan penentuan status terhadap data lokasi yang telah diajukan oleh pengelola lokasi. Proses ini bertujuan untuk memastikan bahwa data lokasi yang masuk telah sesuai dengan ketentuan sebelum ditampilkan dan digunakan di dalam sistem. Proses dimulai ketika sistem menerima permintaan verifikasi berdasarkan ID lokasi. Sistem akan melakukan pengecekan ketersediaan data lokasi. Apabila data lokasi tidak ditemukan, sistem akan memberikan respons kesalahan. Selanjutnya, sistem melakukan validasi terhadap data yang dikirim, meliputi status persetujuan, alasan penolakan, serta identitas karyawan yang melakukan verifikasi. Apabila status verifikasi ditetapkan sebagai Diterima, sistem akan mengubah status lokasi menjadi Aktif, mencatat karyawan yang melakukan verifikasi, serta menghapus alasan penolakan. Sebaliknya, jika status verifikasi ditetapkan sebagai

Ditolak, sistem akan mengubah status lokasi menjadi Nonaktif dan menyimpan alasan penolakan. Setelah proses verifikasi selesai, sistem menyimpan perubahan data dan memberikan respons bahwa status verifikasi lokasi berhasil diperbarui. Melalui fitur ini, sistem memastikan bahwa data lokasi yang tersedia telah melalui proses verifikasi yang sesuai.

Listing 5.17 Potongan Kode Verifikasi Penambahan Data Lokasi Baru

```
public function verifikasi($id, Request $request)
  {
    $lokasi = Lokasi::find($id);
    if (!$lokasi) {
      return response()->json([
        'success' => false,
        'message' => 'Data lokasi tidak ditemukan.'
      ], 404);
    }
    $request->validate([
      'status_acc' => 'required|in:Diterima,Ditolak,Diajukan',
      'alasan' => 'nullable|string|max:255',
      'karyawan_id' => 'nullable|integer|exists:karyawan,karyawan_id',
    ]);
    $lokasi->status_acc = $request->status_acc;
    if ($request->status_acc === 'Diterima') {
      $lokasi->status = 'Aktif';
      $lokasi->karyawan_id = $request->karyawan_id;
      $lokasi->alasan = null;
    } elseif ($request->status_acc === 'Ditolak') {
      $lokasi->status = 'Nonaktif';
      $lokasi->alasan = $request->alasan ?? '-';
      $lokasi->karyawan_id = $request->karyawan_id;
    }
    $lokasi->save();
    return response()->json([
      'success' => true,
      'message' => 'Status verifikasi lokasi berhasil diperbarui.',
      'data' => $lokasi
    ], 200);
  }
```

c. Pengajuan Perubahan Data Lokasi

Fitur Pengajuan Perubahan Data Lokasi digunakan oleh pengelola lokasi untuk mengajukan perubahan terhadap data lokasi yang telah terdaftar di dalam sistem. Proses ini bertujuan agar setiap perubahan data dapat tercatat dan melalui proses persetujuan sebelum diterapkan. Proses dimulai ketika pengelola lokasi mengirimkan daftar kolom data yang ingin diubah beserta nilai baru yang diusulkan. Sistem akan melakukan validasi untuk memastikan bahwa data yang dikirim lengkap dan sesuai, yaitu jumlah kolom yang diubah sebanding dengan jumlah nilai yang diajukan. Apabila validasi gagal, sistem akan memberikan respons kesalahan.

Untuk setiap data yang diajukan, sistem akan mengambil nilai data sebelum perubahan dan membandingkannya dengan nilai yang diusulkan. Selanjutnya, sistem akan mencatat setiap pengajuan perubahan ke dalam tabel histori perubahan lokasi, yang memuat informasi nilai sebelum perubahan, nilai setelah perubahan, serta waktu pengajuan perubahan. Data perubahan yang telah diajukan akan menunggu proses verifikasi oleh karyawan. Setelah seluruh perubahan berhasil dicatat, sistem akan memberikan respons bahwa pengajuan perubahan data lokasi berhasil dilakukan. Melalui fitur ini, sistem dapat memastikan bahwa setiap perubahan data lokasi terdokumentasi dengan baik dan dapat ditelusuri kembali.

Listing 5.18 Potongan Kode Pengajuan Perubahan Data Lokasi

```
public function ubahLokasi(Request $request, $lokasiId)
   {
      $lokasi = Lokasi::findOrFail($lokasiId);
      $kolomList = $request->input('kolom', []);
      $nilaiList = $request->input('nilai', []);
```

Listing 5.18 Potongan Kode Pengajuan Perubahan Data Lokasi (Lanjutan)

```
    if (empty($kolomList) || count($kolomList) !== count($nilaiList)) {
        return response()->json([
            'success' => false,
            'message' => 'Data yang dikirim tidak valid atau tidak lengkap.'
        ], 400);
    }
    $historiAll = [];
    foreach ($kolomList as $index => $namaKolom) {
        $nilaiSesudah = $nilaiList[$index];
        $kategori = KategoriPerubahanLokasi::where('nama_kolom',
$namaKolom)->first();
        if (!$kategori)
            continue;
        $nilaiSebelum = $lokasi->{$namaKolom};
        if ($namaKolom === 'foto') {
            if (is_string($nilaiSesudah)) {
                $decoded = json_decode($nilaiSesudah, true);
                $nilaiSesudah = $decoded ?? $nilaiSesudah;
            }
            $nilaiSebelum = $nilaiSebelum ? json_encode($nilaiSebelum) : null;
            $nilaiSesudah = is_array($nilaiSesudah) ? json_encode($nilaiSesudah) :
$nilaiSesudah;
        }
        $histori = HistoriLokasi::create([
            'lokasi_id' => $lokasi->lokasi_id,
            'karyawan_id' => null,
            'kategori_perubahan_lokasi_id' =>
$kategori->kategori_perubahan_lokasi_id,
            'nilai_sebelum' => $nilaiSebelum,
            'nilai_sesudah' => $nilaiSesudah,
            'tanggal_pengajuan' => now(),
            'tanggal_persetujuan' => null,
            'created_at' => now(),
            'updated_at' => now(),
        ]);
        $historiAll[] = $histori;
    }
    return response()->json([
        'success' => true,
        'message' => 'Perubahan lokasi berhasil diajukan.',
        'histori' => $historiAll,
    ]);}
```

5.2.5 Implementasi Penyewaan Lokasi Acara

Pada subbab ini akan dijelaskan implementasi dari desain proses penyewaan lokasi acara yang telah dibuat pada bab sebelumnya. Setiap penjelasan dilengkapi dengan potongan kode untuk menggambarkan cara kerja program secara lebih jelas. Bagian ini membahas proses pengajuan transaksi acara oleh pengguna, pembuatan kontrak sewa lokasi acara secara otomatis, serta pengelolaan pembayaran transaksi acara yang dilakukan melalui mekanisme verifikasi di dalam sistem.

a. Pengajuan Transaksi Acara

Fitur Pengajuan Transaksi Acara digunakan oleh pengguna untuk mengajukan penyelenggaraan acara pada lokasi tertentu yang tersedia di dalam sistem. Proses ini bertujuan untuk mencatat data acara secara lengkap serta memastikan jadwal acara tidak bertabrakan dengan acara lain pada lokasi yang sama. Proses dimulai ketika pengguna mengirimkan data acara melalui sistem. Sistem akan melakukan validasi terhadap data yang diterima, meliputi informasi lokasi, kategori acara, jadwal pelaksanaan, metode pembayaran, data kontak, jumlah peserta, serta data pendukung seperti proposal dan foto acara. Sistem juga memvalidasi data terkait penyediaan tenant apabila acara membuka tenant. Apabila validasi gagal, data tidak akan diproses lebih lanjut.

Setelah validasi berhasil, sistem akan melakukan pengecekan bentrok jadwal acara pada lokasi yang dipilih. Jika ditemukan jadwal yang bertabrakan dengan acara lain, sistem akan memberikan respons penolakan. Apabila tidak terdapat konflik jadwal, sistem akan menyimpan data transaksi acara ke dalam basis data dengan status Menunggu Konfirmasi. Sistem juga menghitung nilai uang muka (DP)

berdasarkan metode pembayaran yang dipilih, yaitu pembayaran penuh atau cicilan. Selanjutnya, sistem menyimpan data pendukung berupa foto acara dan data tenant lokasi apabila acara membuka tenant. Seluruh proses penyimpanan dilakukan dalam satu transaksi basis data untuk menjaga konsistensi data. Setelah proses berhasil, sistem memberikan respons bahwa pengajuan acara berhasil disimpan beserta informasi acara dan nilai DP. Melalui fitur ini, sistem memastikan bahwa pengajuan acara tercatat dengan baik dan siap untuk proses verifikasi selanjutnya.

Listing 5.19 Potongan Kode Pengajuan Transaksi Acara

```
  public function store(Request $request)
  {
     $validated = $request->validate([
        'lokasi_id' => 'required|integer',
        'pengguna_id' => 'required|integer',
        'kategori_id' => 'required|integer',
        'nama_acara' => 'required|string|max:255',
        'deskripsi' => 'nullable|string',
        'tanggal_mulai' => 'required|date',
        'tanggal_selesai' => 'required|date|after_or_equal:tanggal_mulai',
        'harga' => 'required|numeric',
        'metode_pembayaran' => 'required|in:Penuh,Cicilan',
        'nama_kontak_personal' => 'required|string|max:255',
        'nomor_kontak_personal' => 'required|string|max:20',
        'jumlah_peserta' => 'required|integer|min:1',
        'url_proposal' => 'nullable|file|mimes:pdf,doc,docx|max:5120',
        'foto_acara.*' => 'nullable|image|mimes:jpg,jpeg,png|max:5120',
        'open_tenant' => 'required|boolean',
        'jumlah_tenant' => 'nullable|integer|min:1',
        'tenant_lokasi' => 'required_if:open_tenant,true|array',
        'tenant_lokasi.*.latitude' => 'required_if:open_tenant,true|numeric',
        'tenant_lokasi.*.longitude' => 'required_if:open_tenant,true|numeric',
        'tenant_lokasi.*.kategori' => 'required_if:open_tenant,true|array|min:1',
        'tenant_lokasi.*.kategori.*' =>
'integer|exists:kategori_tenant,kategori_tenant_id',
     ]);
```

Listing 5.19 Potongan Kode Pengajuan Transaksi Acara (Lanjutan)

```
    DB::beginTransaction();
    try {
        $conflict = TransaksiAcara::where('lokasi_id', $validated['lokasi_id'])
            ->whereNotNull('status')
            ->where(function ($query) use ($validated) {
                $query->whereBetween('tanggal_mulai', [$validated['tanggal_mulai'],
$validated['tanggal_selesai']])
                    ->orWhereBetween('tanggal_selesai', [$validated['tanggal_mulai'],
$validated['tanggal_selesai']])
                    ->orWhere(function ($q) use ($validated) {
                        $q->where('tanggal_mulai', '<=', $validated['tanggal_mulai'])
                            ->where('tanggal_selesai', '>=', $validated['tanggal_selesai']);
                    });
            })
            ->exists();
        if ($conflict) {
            return response()->json([
                'success' => false,
                'message' => 'Tanggal acara bentrok dengan acara lain di lokasi ini.',
            ], 422);
        }
        $proposalUrl = $request->hasFile('url_proposal')
            ? $request->file('url_proposal')->store('proposal_acara', 'public')
            : null;
        $dp = $validated['metode_pembayaran'] === 'Cicilan'
            ? $validated['harga'] * 0.5
            : $validated['harga'];
        $acara = TransaksiAcara::create([
            'lokasi_id' => $validated['lokasi_id'],
            'pengguna_id' => $validated['pengguna_id'],
            'kategori_id' => $validated['kategori_id'],
            'nama_acara' => $validated['nama_acara'],
            'deskripsi' => $validated['deskripsi'] ?? null,
            'tanggal_mulai' => $validated['tanggal_mulai'],
            'tanggal_selesai' => $validated['tanggal_selesai'],
            'harga' => $validated['harga'],
            'metode_pembayaran' => $validated['metode_pembayaran'],
            'dp' => $dp,
            'status_acc' => 'Menunggu Konfirmasi',
            'nama_kontak_personal' => $validated['nama_kontak_personal'],
            'nomor_kontak_personal' => $validated['nomor_kontak_personal'],
```

Listing 5.19 Potongan Kode Pengajuan Transaksi Acara (Lanjutan)

```
            'jumlah_peserta' => $validated['jumlah_peserta'],
            'url_proposal' => $proposalUrl,
            'open_tenant' => $validated['open_tenant'],
            'jumlah_tenant' => $validated['jumlah_tenant'] ?? null,
        ]);
        if ($request->hasFile('foto_acara')) {
            foreach ($request->file('foto_acara') as $file) {
                $path = $file->store('foto_acara', 'public');
                FotoAcara::create([
                    'acara_id' => $acara->transaksi_acara_id,
                    'url' => $path,
                ]);
            }
        }
        if ($validated['open_tenant']) {
            foreach ($validated['tenant_lokasi'] as $index => $tenant) {
                $tenantLokasi = TenantLokasi::create([
                    'transaksi_acara_id' => $acara->transaksi_acara_id,
                    'latitude' => $tenant['latitude'],
                    'longitude' => $tenant['longitude'],
                    'nama_tenant' => 'Tenant ' . ($index + 1),
                    'status' => 'Tidak Tersedia',
                ]);
                foreach ($tenant['kategori'] as $kategoriId) {
                    KategoriTenantLokasi::create([
                        'tenant_lokasi_id' => $tenantLokasi->tenant_lokasi_id,
                        'kategori_tenant_id' => $kategoriId,
                    ]);
                }
            }
        }
        DB::commit();
        return response()->json([
            'success' => true,
            'message' => 'Pengajuan acara berhasil disimpan.',
            'data' => [
                'acara' => $acara,
                'dp' => $dp,
            ],
        ], 201);
    } catch (\Exception $e) {
        DB::rollBack();
```

Listing 5.19 Potongan Kode Pengajuan Transaksi Acara (Lanjutan)

```
        return response()->json([
            'success' => false,
            'message' => 'Gagal menyimpan acara: ' . $e->getMessage(),
        ], 500);
    }
}
```

b. Pembuatan Kontrak Sewa Lokasi Acara

Fitur Pembuatan Kontrak Sewa Lokasi Acara digunakan untuk menghasilkan dokumen kontrak sewa lokasi acara secara otomatis berdasarkan data transaksi acara yang tersimpan di dalam sistem. Proses ini bertujuan untuk mendokumentasikan kesepakatan antara penyelenggara acara dan pengelola lokasi secara resmi. Proses dimulai ketika sistem menerima permintaan pembuatan kontrak berdasarkan ID transaksi acara. Sistem akan mengambil data transaksi beserta data terkait, yaitu data lokasi dan data pengguna. Selanjutnya, sistem menyusun data kontrak dan memproses pembuatan dokumen kontrak menggunakan template kontrak sewa lokasi dalam format PDF. Dokumen kontrak yang dihasilkan disimpan ke dalam sistem dengan penamaan file yang sesuai. Setelah itu, sistem akan memperbarui data transaksi dengan mengisi nomor kontrak serta URL kontrak yang dapat diakses oleh pengguna. Sebagai tahap akhir, sistem mengembalikan respons berisi informasi kontrak yang telah berhasil dibuat. Melalui fitur ini, sistem memastikan bahwa proses pembuatan kontrak sewa lokasi acara dilakukan secara otomatis dan terdokumentasi dengan baik.

Listing 5.20 Potongan Kode Pembuatan Kontrak Sewa Lokasi Acara

```
public function generateKontrakLokasi($id)
{
    $transaksi = TransaksiAcara::with(['pengguna', 'lokasi'])->findOrFail($id);
    $view = 'kontrak_sewa_lokasi';
    $data = [
        'transaksi' => $transaksi,
        'lokasi' => $transaksi->lokasi,
        'pengguna' => $transaksi->pengguna,
        'tanggal' => now()->translatedFormat('d F Y'),
    ];
    $pdf = PDF::loadView($view, $data)
        ->setPaper('a4', 'portrait')
        ->setOptions([
            'isHtml5ParserEnabled' => true,
            'isRemoteEnabled' => true,
            'defaultFont' => 'DejaVu Sans',
        ]);
    $folder = storage_path('app/public/kontrak');
    if (!file_exists($folder))
        mkdir($folder, 0755, true);
    $filename = 'Kontrak-Acara-' . $transaksi->transaksi_acara_id . '.pdf';
    $pdf->save($folder . '/' . $filename);
    $transaksi->nomor_kontrak = 'KTR-' . $transaksi->transaksi_acara_id;
    $transaksi->url_kontrak = url('storage/kontrak/' . $filename);
    $transaksi->save();
    return response()->json([
        'success' => true,
        'data' => [
            'url_kontrak' => $transaksi->url_kontrak,
            'nomor_kontrak' => $transaksi->nomor_kontrak
        ]
    ]);
}
```

c. Pembayaran Transaksi Acara

Fitur Pembayaran Transaksi Acara digunakan untuk mengelola perubahan status pembayaran acara berdasarkan hasil verifikasi yang dilakukan oleh pihak terkait. Proses ini bertujuan untuk memastikan bahwa pembayaran acara tercatat dengan status yang sesuai di dalam sistem. Proses dimulai ketika sistem menerima

permintaan perubahan status pembayaran berdasarkan ID pembayaran acara. Sistem akan melakukan pengecekan ketersediaan data pembayaran. Apabila data pembayaran tidak ditemukan, sistem akan memberikan respons kesalahan. Selanjutnya, sistem melakukan validasi terhadap status yang dikirim, yaitu Setujui atau Tolak. Apabila status ditetapkan sebagai Setujui, sistem akan membandingkan tanggal pembayaran dengan tanggal jatuh tempo. Jika pembayaran dilakukan melewati tanggal jatuh tempo, maka status pembayaran akan diubah menjadi Terlambat, sedangkan jika tidak, status akan diubah menjadi Bayar. Jika status ditetapkan sebagai Tolak, sistem akan mengubah status pembayaran menjadi Upload Ulang Pembayaran dan menghapus data bukti pembayaran. Setelah perubahan status dilakukan, sistem menyimpan data pembayaran dan memberikan respons bahwa status pembayaran transaksi acara berhasil diperbarui. Melalui fitur ini, sistem dapat memastikan bahwa proses pembayaran acara berjalan secara terkontrol dan terdokumentasi dengan baik.

Listing 5.21 Potongan Kode Pembayaran Transaksi Acara

```
  public function ubahStatus(Request $request, $id)
  {
     $pembayaran = PembayaranAcara::find($id);
     if (!$pembayaran) {
        return response()->json(['success' => false, 'message' => 'Pembayaran tidak
ditemukan.'], 404);
     }
     $validated = $request->validate([
        'status' => 'required|string|in:Setujui,Tolak'
     ]);
     $tanggalBayar = $pembayaran->tanggal_bayar;
     $tanggalJatuhTempo = $pembayaran->tanggal_jatuh_tempo;
     if ($validated['status'] === 'Setujui') {
```

Listing 5.21 Potongan Kode Pembayaran Transaksi Acara (Lanjutan)

```
            if ($tanggalBayar && $tanggalJatuhTempo) {
                $pembayaran->status = strtotime($tanggalBayar) >
strtotime($tanggalJatuhTempo) ? 'Terlambat' : 'Bayar';
            } else {
                $pembayaran->status = 'Bayar';
            }
        } else {
            $pembayaran->status = 'Upload Ulang Pembayaran';
            $pembayaran->url_bukti_pembayaran = null;
        }
        $pembayaran->save();
        return response()->json(['success' => true, 'message' => 'Status pembayaran
berhasil diperbarui.', 'data' => $pembayaran]);
    }
```

5.2.6 Implementasi Penyewaan Tenant Acara

Pada subbab ini akan dijelaskan implementasi dari desain proses penyewaan tenant acara yang telah dibuat pada bab sebelumnya. Setiap penjelasan dilengkapi dengan potongan kode untuk menggambarkan cara kerja program secara lebih jelas. Bagian ini membahas proses pengajuan sewa tenant oleh pengguna, pembuatan kontrak sewa tenant secara otomatis, serta pengelolaan dan persetujuan pembayaran sewa tenant hingga status penyewaan dinyatakan lunas di dalam sistem.

a. Pengajuan Sewa Tenant

Fitur Pengajuan Sewa Tenant digunakan oleh pengguna untuk mengajukan penyewaan tenant pada lokasi acara yang tersedia di dalam sistem. Proses ini bertujuan untuk mencatat data pengajuan sewa tenant secara lengkap sebelum dilakukan proses verifikasi. Proses dimulai ketika pengguna mengirimkan data pengajuan sewa tenant melalui sistem. Sistem akan melakukan validasi terhadap data

yang diterima, meliputi identitas pengguna, data tenant lokasi, informasi tenant, jumlah pembayaran, serta file pendukung. Apabila proses validasi gagal, sistem akan memberikan respons kesalahan. Selanjutnya, sistem melakukan pengecekan ketersediaan tenant lokasi. Apabila tenant tidak tersedia untuk disewa, sistem akan menolak pengajuan. Jika tenant tersedia, sistem akan menyimpan data pengajuan sewa tenant ke dalam basis data dengan status Menunggu Konfirmasi, termasuk menyimpan file pendukung yang diunggah. Setelah proses penyimpanan selesai, sistem memberikan respons bahwa pengajuan sewa tenant berhasil dikirim. Melalui fitur ini, sistem memastikan bahwa pengajuan sewa tenant tercatat dengan baik dan siap untuk proses verifikasi selanjutnya.

Listing 5.22 Potongan Kode Pengajuan Sewa Tenant

```
public function ajukanSewa(Request $request)
   {
      $validator = Validator::make($request->all(), [
         'tenant_lokasi_id' => 'required|exists:tenant_lokasi,tenant_lokasi_id',
         'pengguna_id' => 'required|exists:pengguna,pengguna_id',
         'nama_tenant' => 'required|string|max:255',
         'kategori' => 'required|string|max:255',
         'no_ktp' => 'nullable|string|max:30',
         'deskripsi' => 'nullable|string',
         'jumlah_dp' => 'nullable|numeric|min:0',
         'total_bayar' => 'required|numeric|min:0',
         'file_pendukung' => 'nullable|file|mimes:pdf,jpg,png|max:2048',
      ]);
      if ($validator->fails()) {
         return response()->json([
            'success' => false,
            'message' => 'Validasi gagal.',
            'errors' => $validator->errors(),
       ], 422);
      }
      $tenantLokasi = TenantLokasi::find($request->tenant_lokasi_id);
```

Listing 5.22 Potongan Kode Pengajuan Sewa Tenant (Lanjutan)

```
        if (!$tenantLokasi || $tenantLokasi->status !== 'Tersedia') {
            return response()->json([
             'success' => false,
             'message' => 'Tenant tidak tersedia untuk disewa.'
          ], 400);
        }
        $pathFile = null;
        if ($request->hasFile('file_pendukung')) {
          $pathFile = $request->file('file_pendukung')->store('file_pendukung',
'public');
        }
        $tenantDaftar = TenantDaftar::create([
          'tenant_lokasi_id' => $request->tenant_lokasi_id,
          'pengguna_id' => $request->pengguna_id,
          'nama_tenant' => $request->nama_tenant,
          'kategori' => $request->kategori,
          'no_ktp' => $request->no_ktp,
          'deskripsi' => $request->deskripsi,
          'status' => 'Menunggu Konfirmasi',
          'jumlah_dp' => $request->jumlah_dp,
          'total_bayar' => $request->total_bayar,
          'file_pendukung' => $pathFile,
          'url_kontrak' => null,
        ]);
        return response()->json([
          'success' => true,
          'message' => 'Pengajuan sewa tenant berhasil dikirim.',
          'data' => $tenantDaftar,
        ], 201);
    }
```

b. Pembuatan Kontrak Sewa Tenant

Fitur Pembuatan Kontrak Sewa Tenant digunakan untuk menghasilkan dokumen kontrak sewa tenant secara otomatis berdasarkan data pengajuan sewa tenant yang tersimpan di dalam sistem. Proses ini bertujuan untuk mendokumentasikan kesepakatan sewa tenant secara resmi antara penyewa dan penyelenggara acara. Proses dimulai ketika sistem menerima permintaan pembuatan

kontrak berdasarkan ID pengajuan tenant. Sistem akan melakukan pengecekan ketersediaan data pengajuan tenant. Apabila data tidak ditemukan, sistem akan memberikan respons kesalahan. Selanjutnya, sistem menyusun nomor kontrak secara otomatis berdasarkan tahun, bulan, dan ID pengajuan tenant. Sistem kemudian memproses pembuatan dokumen kontrak menggunakan template kontrak sewa tenant dalam format PDF dengan memuat informasi pengajuan tenant dan tanggal pembuatan kontrak. Dokumen kontrak yang dihasilkan disimpan ke dalam sistem, kemudian sistem memperbarui data pengajuan tenant dengan mencatat URL kontrak. Sebagai tahap akhir, sistem memberikan respons bahwa kontrak sewa tenant berhasil dibuat beserta informasi file kontrak yang dapat diakses. Melalui fitur ini, sistem memastikan bahwa proses pembuatan kontrak sewa tenant dilakukan secara otomatis dan terdokumentasi dengan baik.

Listing 5.23 Potongan Kode Pembuatan Kontrak Sewa Tenant

```
public function generateKontrakTenant($id)
   {
      $tenantDaftar = TenantDaftar::with([
         'tenantLokasi.transaksiAcara',
         'pengguna',
      ])->find($id);
      if (!$tenantDaftar) {
         return response()->json([
            'success' => false,
            'message' => 'Data tenant daftar tidak ditemukan.'
         ], 404);
      }
      $tahun = now()->year;
      $bulan = now()->format('m');
      $idFormatted = str_pad($tenantDaftar->tenant_daftar_id, 4, '0',
STR_PAD_LEFT);
      $nomorKontrak = "KST-$tahun/$bulan/$idFormatted";
```

Listing 5.23 Potongan Kode Pembuatan Kontrak Sewa Tenant (Lanjutan)

```
    $data = [
        'tenantDaftar' => $tenantDaftar,
        'tanggal' => now()->translatedFormat('d F Y'),
        'nomorKontrak' => $nomorKontrak,
    ];
    $pdf = PDF::loadView('kontrak_sewa_tenant', $data)
        ->setPaper('a4', 'portrait')
        ->setOptions([
            'isHtml5ParserEnabled' => true,
            'isRemoteEnabled' => true,
            'defaultFont' => 'DejaVu Sans',
        ]);
    $filename = 'Kontrak-Tenant-' . $tenantDaftar->tenant_daftar_id . '.pdf';
    $path = storage_path('app/public/kontrak/' . $filename);
    $pdf->save($path);
    $tenantDaftar->url_kontrak = 'storage/kontrak/' . $filename;
    $tenantDaftar->save();
    return response()->json([
        'success' => true,
        'message' => 'Kontrak sewa tenant berhasil dibuat.',
        'data' => [
            'url_kontrak' => url('storage/kontrak/' . $filename),
            'file_name' => $filename,
        ],
    ]);
}
```

c. Pembayaran Sewa Tenant

Fitur Pembayaran Sewa Tenant digunakan untuk menyetujui pembayaran sewa tenant yang telah diajukan oleh penyewa. Proses ini bertujuan untuk memastikan bahwa pembayaran sewa tenant tercatat dengan benar dan status penyewaan diperbarui sesuai dengan kondisi pembayaran. Proses dimulai ketika sistem menerima permintaan persetujuan pembayaran berdasarkan ID pembayaran tenant. Sistem akan mengubah status pembayaran menjadi Bayar dan menyimpan perubahan tersebut. Selanjutnya, sistem akan melakukan pengecekan terhadap seluruh

pembayaran yang terkait dengan pengajuan sewa tenant yang sama. Apabila seluruh pembayaran telah berstatus Bayar, sistem akan memperbarui status pengajuan sewa tenant menjadi Disetujui (Lunas). Setelah proses selesai, sistem memberikan respons bahwa pembayaran telah disetujui beserta informasi status terbaru dari pengajuan sewa tenant. Melalui fitur ini, sistem dapat memastikan bahwa proses pembayaran sewa tenant berjalan dengan baik dan terdokumentasi.

Listing 5.23 Potongan Kode Pembayaran Sewa Tenant

```
public function setujui($id)
   {
      $pembayaran = PembayaranTenant::findOrFail($id);
      $pembayaran->status = 'Bayar';
      $pembayaran->save();

      $tenantDaftarId = $pembayaran->tenant_daftar_id;
      $adaBelumSetujui = PembayaranTenant::where('tenant_daftar_id',
$tenantDaftarId)
         ->where('status', '!=', 'Bayar')
         ->exists();

      if (!$adaBelumSetujui) {
         $tenant = TenantDaftar::find($tenantDaftarId);
         if ($tenant) {
            $tenant->status = 'Disetujui (Lunas)';
            $tenant->save();
         }
      }
      return response()->json([
         'message' => 'Pembayaran disetujui',
         'tenant_status' => isset($tenant) ? $tenant->status : null
      ]);
   }
```

5.2.7 Implementasi Rekomendasi Lokasi Acara Menggunakan Metode TOPSIS

Metode *Technique for Order Preference by Similarity to Ideal Solution* (TOPSIS) merupakan salah satu metode pengambilan keputusan multikriteria yang digunakan untuk menentukan alternatif terbaik berdasarkan kedekatannya dengan solusi ideal positif dan kejauhan dari solusi ideal negatif. Dalam sistem ini, metode TOPSIS diterapkan untuk memberikan rekomendasi lokasi acara yang paling sesuai dengan kebutuhan pengguna berdasarkan beberapa kriteria yang ditentukan.

Potongan kode pada Listing 5.24 merupakan implementasi endpoint rekomendasi lokasi acara yang memanfaatkan metode TOPSIS. Proses dimulai ketika sistem menerima permintaan rekomendasi lokasi dari pengguna yang berisi kriteria seperti tanggal mulai dan tanggal selesai acara, jumlah peserta, anggaran lokasi, serta preferensi jenis lokasi. Seluruh data masukan tersebut terlebih dahulu divalidasi untuk memastikan kesesuaian format dan batasan nilai yang ditetapkan oleh sistem.Setelah proses validasi berhasil, data kriteria yang telah lolos validasi diteruskan ke kelas layanan TopsisLokasiService melalui pemanggilan metode recommend(). Kelas layanan ini bertanggung jawab menjalankan tahapan perhitungan TOPSIS, mulai dari pengolahan data lokasi sebagai alternatif, pembobotan kriteria, normalisasi nilai, hingga perhitungan nilai preferensi untuk setiap lokasi. Hasil perhitungan tersebut berupa daftar lokasi yang telah diurutkan berdasarkan tingkat rekomendasi tertinggi.

Selanjutnya, sistem mengembalikan respons dalam format JSON yang memuat status keberhasilan proses, jumlah lokasi yang direkomendasikan, serta data peringkat lokasi hasil perhitungan TOPSIS. Dengan implementasi ini, sistem mampu

memberikan rekomendasi lokasi acara secara objektif dan terstruktur sebagai dasar pengambilan keputusan bagi pengguna.

Listing 5.24 Potongan Kode Rekomendasi Lokasi Acara Menggunakan Metode TOPSIS

```
public function lokasi(Request $request, TopsisLokasiService $service)
{
    $validated = $request->validate([
        'tanggal_mulai' => 'required|date',
        'tanggal_selesai' => 'required|date|after:tanggal_mulai',
        'jumlah_peserta' => 'required|integer|min:1',
        'budget_lokasi' => 'nullable|numeric|min:0',
        'prefer_jenis_lokasi' => 'nullable|in:Dalam,Luar',
    ]);
    $ranking = $service->recommend($validated);
    return response()->json([
        'success' => true,
        'count' => $ranking->count(),
        'ranking' => $ranking,
    ]);
}
```

5.2.8 Implementasi Pencarian Tenant Menggunakan Rules-Based Filtering

Metode rules-based filtering merupakan pendekatan pencarian berbasis aturan yang digunakan untuk menyaring data berdasarkan kriteria tertentu yang telah ditetapkan. Dalam sistem ini, metode rules-based filtering diterapkan untuk membantu pengguna menemukan tenant yang sesuai dengan kebutuhan acara berdasarkan kategori tenant dan keterkaitan dengan acara tertentu.

Potongan kode pada Listing 5.25 merupakan implementasi fungsi pencarian tenant menggunakan pendekatan rules-based filtering. Proses diawali dengan penerimaan data masukan dari pengguna berupa kategori tenant yang diinginkan serta ID acara sebagai parameter opsional. Sistem terlebih dahulu melakukan validasi

untuk memastikan bahwa kategori tenant dan data acara yang dikirimkan benar-benar tersedia di dalam basis data.

Setelah validasi berhasil, sistem membangun query untuk mengambil data tenant lokasi dengan status Tersedia serta memuat relasi transaksi acara dan kategori tenant. Selanjutnya, sistem menerapkan aturan penyaringan dengan memastikan bahwa tenant yang diambil memiliki kategori yang sesuai dengan kategori tenant yang dipilih pengguna. Apabila parameter acara dikirimkan, sistem juga akan membatasi hasil pencarian hanya pada tenant yang terkait dengan acara tersebut.

Data tenant yang diperoleh kemudian diproses dan dipetakan ulang untuk menampilkan informasi penting, seperti identitas tenant lokasi, nama tenant, harga, status, kategori tenant, serta nama acara yang terkait. Selanjutnya, sistem mengembalikan hasil pencarian dalam bentuk respons JSON yang memuat status keberhasilan proses, pesan informasi, serta daftar tenant hasil penyaringan. Dengan implementasi ini, sistem mampu menyediakan rekomendasi tenant yang relevan secara cepat dan terarah berdasarkan aturan yang telah ditentukan.

Listing 5.25 Potongan Kode Pencarian Tenant Menggunakan Rules-Based Filtering

```
public function recommendTenant(Request $request)
{
        $request->validate([
                'kategori_tenant_id' =>
'required|integer|exists:kategori_tenant,kategori_tenant_id',
                'acara_id' =>
'nullable|integer|exists:transaksi_acara,transaksi_acara_id'
        ]);
```

Listing 5.25 Potongan Kode Pencarian Tenant Menggunakan Rules-Based Filtering (Lanjutan)

```
	$query = TenantLokasi::with(['transaksiAcara', 'kategoriTenant'])
->where('status', 'Tersedia');
	$query->whereHas('kategoriTenant', function ($q) use ($request) {
$q->where('kategori_tenant.kategori_tenant_id', $request->kategori_tenant_id);
	});
	if ($request->acara_id) {
$query->where('tenant_lokasi.transaksi_acara_id', $request->acara_id);
	}
	$tenant = $query->get()->map(function ($t) {
	return [
		'tenant_lokasi_id' => $t->tenant_lokasi_id,
		'nama_tenant' => $t->nama_tenant,
		'harga' => $t->harga,
		'status' => $t->status,
		'kategori' => $t->kategoriTenant->pluck('nama'),
		'acara' => $t->transaksiAcara->nama_acara ?? '-',
		];
	});
	return response()->json([
		'success' => true,
		'message' => 'Rekomendasi tenant berhasil diambil (Rules-Based
Filtering).',
		'data' => $tenant
	]);
}
```